

FYS-STK4155: Project 1

Github repository: https://github.com/nillescorner/FYS-STK4155-Group/tree/main/Project_1

Pernille Xu Amundsen, Maria Andersen, Hannah Westgaard

(Dated: October 7, 2026)

An algorithm that uses training data to "learn" patterns to thereafter make accurate predictions about new data is the essence of machine learning. Thus, optimizing a model's performance on training data is fundamental as the better the model performs, the more accurate predictions it can make on new data, which is its ultimate use case.

In order to pick which models and methods are the most optimal for its use case, one must study the different ones, which is exactly what we've done in this project. To choose the correct model, we used applied linear algebra in python to compare the different regression methods: Ordinary Least Squares, Ridge and Lasso. We also studied two resampling techniques: bootstrap and cross-validation together with scikit-learn's own library. We also used different optimization methods, such as gradient descent, momentum, AdaGrad, RMSProp, Adam and stochastic gradient descent. Based on cross-validation hyperparameter tuning, models were trained and tested on the whole dataset. The resulting best models with mean square errors on the order of 2×10^{-2} did not manage to reproduce the true Runge function.

I. INTRODUCTION

Artificial intelligence (AI) has in the past years become an integral part of modern society, especially after the launch of ChatGPT in November 2022 [1]. AI in itself, is the technology that enables computers and machines to simulate human learning, comprehension, problem solving, decision making, creativity and autonomy [2]. AI can be divided into different subsets, where the most dominating subset is machine learning. Machine learning provides the infrastructure of most modern AI systems as it uses algorithms that can "learn" patterns in training data, to later on make accurate predictions about new data. Due to its vast use cases, as there is no limit on what the training data can be, many industries such as science, technology, healthcare and finance have become somewhat reliant on machine learning models. It is therefore important that these models are reliable and in their most optimal state [3].

For a model to be reliable, it must perform consistently well on new data. It is therefore important that the model actually learns the patterns in the training data instead of just memorizing it. When a model fits the data too closely to the training set, it becomes overfitted, and it cannot predict patterns in new data. Overfitting makes the model unreliable since it only performs well on training data and can be the result of many factors such as model complexity, lack of training data, etc. A model also needs to be trained with a suitable optimizer for its use case. Optimizers are algorithms that search for the parameter θ that minimizes a model's cost function, and they differ in how they use the gradient of that cost function to decide each step. We will be using plain gradient descent (GD), stochastic gradient descent (SGD) and the more advanced optimizers momentum, AdaGrad, RMSProp and Adam. GD works by taking a fixed-size step in the direction of steepest descent, while the more advanced methods decide the step size by using information about past gradients. SGD estimates the

gradient from a random subset of the data at each step rather than the full dataset, trading a noisier direction for a much cheaper update. This is necessary when the dataset becomes too large for exact gradients to be computed cheaply at every step. Different optimizers can converge to slightly different parameters, or converge at different speeds, for the same problem. It is therefore useful to try different optimizers to see which gives the best result for its use case [4].

Our target data in this project are generated in vector $\mathbf{y} \in \mathbb{R}^n$ as:

$$\mathbf{y} = f(\mathbf{x}) + \boldsymbol{\varepsilon}, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2) \quad (.1)$$

where $\mathbf{x} \in \mathbb{R}^n$ is our input data, f is typically an unknown function, and $\boldsymbol{\varepsilon}$ is independent noise with a mean of 0 and a variance of σ^2 . In this project we use the Runge function:

$$f(x) = \frac{1}{1 + 25x^2} \quad (.2)$$

as f , where the input data x_i are uniformly selected random data points in the interval $[-1, 1]$. The Runge function is interesting since it is a smooth and well-behaved function, but if it is interpolated with high-degree polynomials at equally spaced points, the resulting curve will oscillate at the edges of the interval [5].

In section II we will introduce the different methods and optimizers used throughout this project. We will describe how they were implemented in our code as well as what their purpose was. We will then display the results from the different methods and optimizers in section III, using a combination of figures and tables. We will also interpret, analyze, and discuss the results gathered from our code in section IV. In section V we will present our main findings and interpretations, as well as reflect over possible improvements for further research. We have also

attached an Appendix: A which contains extra calculations, the Large Language Models (LLM Tools) used for this project and additional figures. A link to the GitHub repository is also attached to this report beneath the title.

II. METHODS

Unless stated otherwise, throughout this project the data are generated with the number of data points n set to 100 and the noise $\sigma = 0.1$. The data points are uniformly selected data points using the 2026 as the seed for the random state. This is implemented so that the data is reproducible.

A. Linear Regression

When we work with linear regression models, our task is to construct an approximation to the function f from the data. The approximation can be written using a model that is linear in the parameters:

$$\tilde{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta}, \quad \tilde{y}_i = \sum_{j=0}^{p-1} x_{ij}\theta_j = \mathbf{X}_{i,*}\boldsymbol{\theta} \quad (\text{A.1})$$

where $\boldsymbol{\theta} \in \mathbb{R}^p$ are the model parameters, $\mathbf{X} \in \mathbb{R}^{n \times p}$ is the design matrix and $\mathbf{X}_{i,*}$ represents the i th row. The word *linear* refers to the parameters and not to the features, which means that nothing prevents the columns of the design matrix $\mathbf{X}$ from being non-linear functions of some underlying variable [4]. For a polynomial with degree $p - 1$ in the input $\mathbf{x} \in \mathbb{R}^n$, the design matrix can be built as:

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{p-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{p-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^{p-1} \end{bmatrix}$$

All the linear regression methods use this model to predict $\tilde{\mathbf{y}}$ but differ in the way they derive $\boldsymbol{\theta}$ since their cost functions differ.

1. Cost Functions and Measures of Quality

The ability of a model or method to reproduce the data it has been trained on is measured using a cost function, also called a loss or error function. The cost function differs depending on which method and model was used. In machine learning, cost functions are scalars that depend on a vector or a matrix of parameters. We therefore need derivatives with respect to vectors and matrices to minimize them [4]. The model parameters $\boldsymbol{\theta}$ can be found by minimizing the cost function $C(\boldsymbol{\theta})$ on the training data.

When we compare the different linear regression methods, we evaluate them using the mean squared error (MSE):

$$MSE(\mathbf{y}, \tilde{\mathbf{y}}) = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2 \quad (\text{A.2})$$

The MSE is a measure of how much the predictions $\tilde{\mathbf{y}}$ differ from the target data $\mathbf{y}$. The difference is squared to penalize larger errors more and to avoid positive and negative errors from cancelling out one another.

In addition to the MSE, we also use the *coefficient of determination*

$$R^2(\mathbf{y}, \tilde{\mathbf{y}}) = 1 - \frac{\sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2}{\sum_{i=0}^{n-1} (y_i - \bar{y})^2}, \quad \bar{y} = \frac{1}{n} \sum_{i=0}^{n-1} y_i$$

which compares the model against the best constant prediction. An $R^2 = 1$ indicates a perfect fit, whereas $R^2 = 0$ means that the model does no better than predicting the mean and $R^2 < 0$, which is entirely possible on test data, means that it does worse. The R^2 score can also be written as $1 - \text{MSE}/\text{Var}[\mathbf{y}]$ [4]

2. Ordinary Least Squares (OLS) Regression

For Ordinary Least Squares (OLS), the cost function is the MSE (Eq. A.2) given as:

$$C(\boldsymbol{\theta}) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2 \quad (\text{A.3})$$

Its Hessian $\frac{2}{n} \mathbf{X}^T \mathbf{X}$ is positive semi-definite so the cost function is convex and any stationary point is a global minimum. The differentiation of the cost function and its Hessian can be found on p.15 in [4]. The optimal parameter is derived by setting the gradient of the cost function to zero, this calculation gives the normal equations $\mathbf{X}^T \mathbf{X}\boldsymbol{\theta} = \mathbf{X}^T \mathbf{y}$ with solution:

$$\hat{\boldsymbol{\theta}}_{\text{OLS}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (\text{A.4})$$

whenever the inverse exists. For high polynomial degrees the columns x^k of $\mathbf{X}$ become very similar to each other over the interval $[-1, 1]$. The design matrix is then near rank deficient, which is a dangerous case since small changes in the data can give very large changes in $\hat{\boldsymbol{\theta}}$. We therefore use the Moore-Penrose pseudo-inverse, which provides a substitute for the inverse via the SVD, in order to work around nearly linearly dependent features [4].

3. Ridge Regression

Ridge regression augments the cost function for OLS (Eq. A.3) by adding a penalty on the size of the parameters. The cost function is thus:

$$C_{\text{ridge}}(\boldsymbol{\theta}) = \frac{1}{n}(\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^T(\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) + \lambda\|\boldsymbol{\theta}\|_2^2, \quad \lambda \geq 0 \quad (\text{A.5})$$

but we drop the $1/n$ factor so that λ enters the solution directly without a factor of n , thus (Eq. A.5) becomes:

$$C_{\text{ridge}}(\boldsymbol{\theta}) = (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^T(\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) + \lambda\|\boldsymbol{\theta}\|_2^2, \quad \lambda \geq 0 \quad (\text{A.6})$$

Following the same procedure as for OLS, we set the gradient of the cost function to zero and derive the normal equation $(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})\boldsymbol{\theta} = \mathbf{X}^T\mathbf{y}$ and the optimal parameters:

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y} \quad (\text{A.7})$$

where $\mathbf{I}$ is the $p \times p$ identity matrix. From (Eq. A.7) we see that the penalty parameter λ is added to every eigenvalue of $\mathbf{X}^T\mathbf{X}$. For $\lambda = 0$ this is just the OLS (Eq. A.4) while $\lambda > 0$ makes the matrix $\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}$ positive definite however dependent the columns of $\mathbf{X}$ may be [4]. Unlike OLS which can be severely affected by small changes in the data, Ridge contains the penalty term $\lambda\|\boldsymbol{\theta}\|_2^2$ which shrinks the coefficients and thereby makes the model more stable.

The SVD substitutes $\mathbf{X} = \mathbf{U}\Sigma\mathbf{V}^T$ and the optimal parameter can be written as:

$$\tilde{\mathbf{y}}_{\text{Ridge}} = \mathbf{X}\hat{\boldsymbol{\theta}}_{\text{Ridge}} = \sum_{i=0}^{p-1} \mathbf{u}_i \mathbf{u}_i^T \frac{s_i^2}{s_i^2 + \lambda} \mathbf{y}, \quad (\text{A.8})$$

where s_i are the singular values of $\mathbf{X}$ and $\mathbf{u}_i$ are the columns of $\mathbf{U}$. For OLS, there is no penalty parameter λ so every factor equals one. For Ridge this is shrinkage factor $s_i^2/(s_i^2 + \lambda)$ which lies between zero and one. Directions with nearly linear dependent features are therefore suppressed the most. The sum of these factors is the effective degree of freedom:

$$df(\lambda) = \sum_{i=0}^{p-1} \frac{s_i^2}{(s_i^2 + \lambda)}$$

It tells us how many parameters Ridge regression uses (more on this in Sec 1.22 and 3.8 in [4]).

4. Lasso Regression

We've earlier looked at OLS and Ridge regression, and we will now tackle Lasso regression. This method can

look a lot like Ridge regression, as it also uses a penalty, but here we use a ℓ_1 penalty instead of ℓ_2 . This is not a large change to the cost function, but there is a large difference in the result. By doing this, Lasso shrinks the coefficients to exactly zero, where Ridge only minimises them toward zero. The cost function for Lasso is written as

$$C_{\text{Lasso}}(\boldsymbol{\theta}) = \frac{1}{n}\|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda\|\boldsymbol{\theta}\|_1, \quad \|\boldsymbol{\theta}\|_1 = \sum_{j=1}^p |\theta_j|, \quad (\text{A.9})$$

where λ decides the strength of the penalty.

Lasso, different from Ridge, automatically identifies irrelevant features, and discards them by setting their coefficients to zero. Adding this penalty, shrinks the coefficients which again reduces the variance.

When we have the ℓ_1 penalty, $|\theta_j|$ won't be differentiable at $\theta_j = 0$ because of the penalty term. At $\theta_j = 0$, there will be a sharp corner, so the curve will look like a "V" shape. This will cause the slope to jump from -1 to +1, instead of having a smooth transition. To solve this problem we will need to use a proximal operator for the ℓ_1 term, which will smooth out the non-smooth coordinate. Instead of using the classic gradient descent method, we can use a coordinate descent method. This takes the gradient of each point exactly, using the coordinates own curvature rather than the global step size that gradient descent uses.

B. Splitting and Scaling of Data

As stated earlier, machine learning uses algorithms that can "learn" patterns in training data to later make accurate predictions about new data. When we explore the regression methods, resampling techniques and the different optimizers, we split the data in order to get an understanding of what the method has learnt vs what it has memorized. This shows us how well a method can generalize and how well it can make predictions about data it has never seen. For this project the data are split so that 4/5 is used for training data and 1/5 is used for testing. This was chosen because it is an accepted rule of thumb [6].

Scaling of the data is particularly important for penalised regression. For Ridge and Lasso regression, we showed that the cost functions contain the penalties ℓ_1 and ℓ_2 which treat all coefficients equally, disregarding the fact that the units of the features (columns) may differ. An example of this would be a model with features that measures a person's height, where one does so in millimetres and the other in metres. The fitted coefficients of the features measured in metres would be penalized 10^3 more for the ℓ_1 and 10^6 for ℓ_2 . It is there important that each feature is standardised so that each column of the design matrix has mean 0 and standard deviation 1 so the coefficients are penalized at the same magnitude [4].

The mean and standard deviation is only computed on the training data which is then applied to the test data. If we computed these terms before on the full data set before splitting, information would leak from the test set into the training procedure which would produce more optimistically biased estimates.

The design matrix contains a column of ones for the intercept. This is transformed to a column of zeros after standardisation which does not contribute to the fit. Therefore we centre $\mathbf{y}$ by subtracting the mean of the training targets, so the intercept is given by this mean. This makes it so the intercept is not penalised.

Since OLS does not contain this penalty, scaling is not as important. On p. 113 of [4] we can see that OLS is equivariant under rescaling of the columns whereas the penalised methods are not. Therefore it is not necessary to scale when doing OLS regression, but we did it to give a fair comparison between OLS, Ridge and Lasso regression so that all methods use the preprocessed data.

C. The Bias-Variance Tradeoff

The cost we wish to decompose in the bias-variance tradeoff is the expected mean squared error:

$$\mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2$$

We do this by inserting $\mathbf{y} = f(\mathbf{x}) + \varepsilon$ and adding and subtracting the average prediction over the training sets $\mathbb{E}[\tilde{\mathbf{y}}]$ [4]. The full calculation can be viewed in the Appendix (A). The result is a sum of three terms:

$$\mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] = \underbrace{\frac{1}{n} \sum_i (f_i - \mathbb{E}[\tilde{\mathbf{y}}])^2}_{\text{Bias}[\tilde{\mathbf{y}}]^2} + \underbrace{\frac{1}{n} \sum_i (\tilde{y}_i - \mathbb{E}[\tilde{\mathbf{y}}])^2}_{\text{Var}[\tilde{\mathbf{y}}]} + \sigma^2 \quad (\text{C.1})$$

where $f_i = f(x_i)$ and σ^2 is the variance of the noise. The expectation value $\mathbb{E}$ is taken over many different training sets. Since f is usually an unknown function, we replace f_i by y_i when we compute the bias. Thus we can rewrite $y_i = f_i + \varepsilon_i$, and the computed term contains the irreducible error σ^2 .

The **bias** measures how far the average prediction taken over different training sets is from the true function. A high bias means that the model cannot capture the patterns in the data since it is too simple (i.e. it underfits), this typically happens for models with low complexity. The **variance** measures how much the prediction changes when the training set is redrawn. A high variance means that the model is sensitive to small changes in the training data and typically happens for more complex models. The term σ^2 is the **irreducible error** which is the variance of the noise itself. No matter how good a model is, it cannot predict a random term, so σ^2 is the

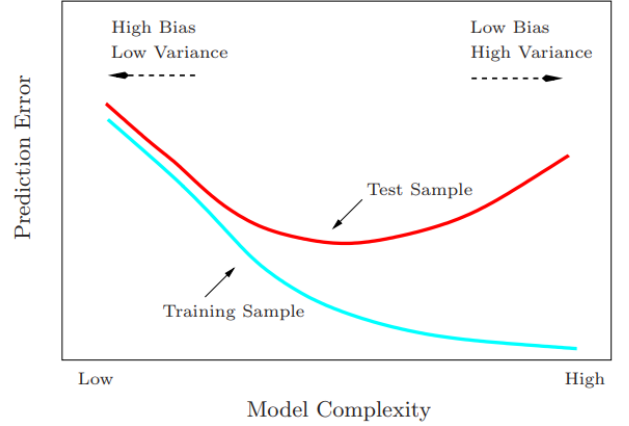


Figure 1: Test and training error as a function of model complexity. Taken from Hastie et al. [7]

lower limit for the expected test error. σ^2 is often called the noise floor and acts as limit of how well a model can perform [4].

From (Eq. C.1) we see that the error is the sum of these three terms. σ^2 is a set value by the data, so the model complexity can only affect the bias and variance. Typically reducing one means increasing the other, which is the bias-variance tradeoff and can be shown in Figure 1. Our goal is therefore to find the polynomial degree that minimises the sum of the bias and variance in order to derive the lowest test MSE.

D. Resampling Methods

When we are working with one set of data, the train-test split will only give an estimate of the test error for that particular split. It is therefore dependent on which data ended up in the training split and gives no information of how the model would change if the training set looked different. Resampling methods work around this by using the same data to make multiple training sets and refitting the model for each of these. In this project we have used bootstrap and k -fold cross-validation.

1. Bootstrap

To study the bias and variance of the model as discussed in the section above, we need to know how the predictions vary over many different training sets. Instead of generating these directly, since we do not know how the data was distributed, we use the bootstrap resampling method.

The bootstrap is carried out as follows: We randomly draw n points from the training data in order to form a bootstrap sample. The same point can be drawn more than once. We train the model on the bootstrap sample and use it to predict the values at the test points.

This is repeated $n_{\text{bootstraps}}$ times, in our case this was 100. And we get 100 different predictions for each test point. We can then derive the bias and variance from these predictions.

Since bootstrap resampling uses the same data set, on average, only 63.2% are unique observations whereas the remainders are duplicates. This leaves 36.8% as *out-of-bag* observations and the training set will be smaller.

2. k -fold Cross-validation

In this project, two different methodologies were used for cross-validation, guided by different sources of information. For the initial implementation the data was split in k -folds with no training set, guided by the lecture book[4]. The implementation for tuning hyperparameters, although remaining largely the same, implemented a test-train split, with k folds on the training data, guided by the scikit-learn user guide [8]. Aside from the splitting of data, the initial implementation in IID 2 a is reused in the second implementation IID 2 b

a. Cross validation k -fold cross validation reduces the dependence on a single train-test split by using the whole data set. The data is divided into k subsets of roughly equal size called folds. Each of these folds takes turns being the test set while the remaining $k - 1$ folds serve as the training set. Each observation is thus used for training $k - 1$ times and for testing exactly once. This improves the ability to develop a model when the data is sparse, with the downside that it becomes more computationally expensive. As the dataset in this project is small, this is of no issue.

The procedure is as follows. We first shuffle the data set at random, then split it into k folds. For each fold, we set it aside and fit the model on the remaining folds and evaluate it on the fold that was set aside. We then record the score, also known as the cost function, and discard the model. The mean of the k scores give the cross-validation MSE. The standard deviation of the k scores shows how much the error varies between the folds [4].

When working with using k -fold cross-validation it is important that any preprocessing of the data must be performed inside the cross-validation loop. If we scale the data set before the loop, each training fold can see the mean and variance of its own test fold (i.e. data leak), causing the estimate of the error to be too optimistic.

b. Cross validation with hyperparameters In the case where one wants to do a train-test split with cross-validation on the training split as in fig. 2, the main procedure follows as in IID 2 a, but using the training data only for the cross-validation, giving the averaged MSE per cross-validation on the training set.

As there are many different estimators in the machine learning world, such as OLS, Ridge and Lasso, one needs

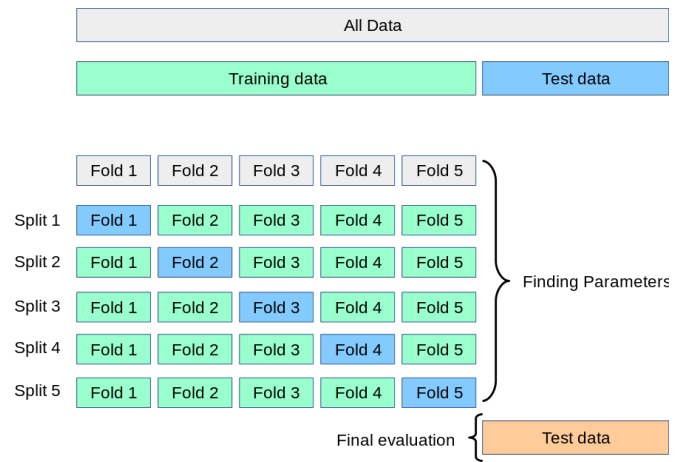


Figure 2: Visualisation of division of data using k -fold cross-validation[8]

a way to compare them. They have different mathematical descriptions, with Ridge and Lasso having no direct point of comparison. Out of a selection of models, one can tune each parameter, so that the best model yields the best score out of possible parameters.

The search of such parameters is called tuning the hyperparameters, which in our case for Ridge, Lasso and OSL would be the degree, and for Ridge and Lasso the penalty λ . We apply the same cross-validation loop, but now over a grid of these hyperparameters. For each combination, the hyperparameters yielding the minimised cost function are stored and used to train a model on the whole train-split. We use this same model to calculate the test MSE using the test-split.

To validate, we compare the train and test MSEs. A test MSE that is close to the train MSE indicates that the model generalizes well with little overfitting to the training data [8]. While the best model, this does not mean the cheapest. It is typical to then chose the best cheapest model, meaning for example fewer degrees, as long as the score of the model is within one standard error above the error of the best model [7].

MSE decomposition using the training set was also attempted as per the bootstrap method of Bias-Variance decomposition. Additionally, a resampling strategy using k -folds was also developed, where the splitting is as described in IID 2 a. Final fits were done on the whole dataset.

E. Optimizers

We have now looked at different ways of finding the optimal parameters for our model, but how can minimize the cost of these parameters? To do this we will use different types of *optimizers*, and study how well different optimizer methods perform in relation to each other. What an optimizer generally does is find the optimal pa-

parameter that minimises the cost function $C(\theta)$, but there are different ways to find a minimum.

1. Gradient descent code, with analytical gradients and automatic differentiation

Gradient descent is an optimization method used to optimize the model which describes our data. For any model we have, to describe a data set, there will be a cost function, $C(\mathbf{X}, g(\theta))$, that tells us how far off from the truth our model is. Our goal is to minimize the cost function, so that our model is as close to the data set as possible.

Gradient descent works by finding the gradient of a given point of the cost function, and then moving a *small* increment in the negative direction of the gradient, so down toward the minimum. After we've moved a small step down, we calculate the gradient again and repeat the process. This process repeats until the slope of the gradient gets small enough for us to assume that we're on, or close to the minimum. We can write the method of gradient descent as

$$\theta_{k+1} = \theta_k - \gamma \nabla_{\theta} C(\theta_k), \quad (\text{E.1})$$

where θ is the optimal parameter, k is the iteration number and γ decides the length of the step we take between each time we calculate the gradient. In machine learning, we call this the *learning rate*. The learning rate needs to be small enough, so that we don't overshoot the minimum by taking too large steps. But we also need to make sure it's not so small that our program efficiency is effected. In figure 3 an illustration of how the learning rate as a step size, decides where the program stops when approaching the minimum.

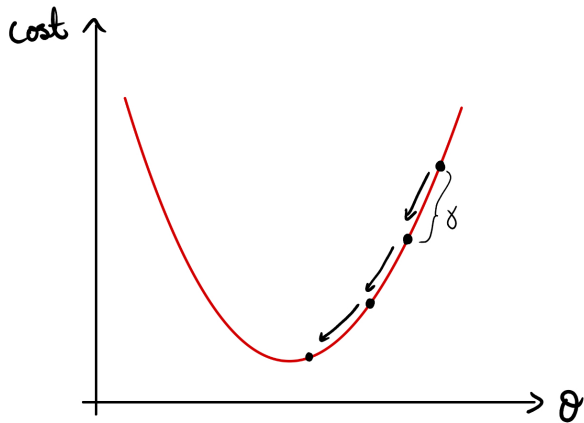


Figure 3: Illustration of how gradient descent find the minimum with the learning rate γ as the step size.

To find the limit of how large the learning rate can be

$\gamma_{\max}$ we use the equation

$$\gamma_{\max} = \frac{2}{\lambda_{\max}(\mathbf{H})}, \quad (\text{E.2})$$

where $\lambda_{\max}(\mathbf{H})$ is the is the largest eigenvalue of the Hessian, given as

$$\mathbf{H} = \frac{2}{n} \mathbf{X}^T \mathbf{X} + 2\lambda \mathbf{I}, \quad (\text{E.3})$$

where the factor with the penalty λ is used for Ridge, and for OLS this factor disappears, as there is no penalty.

The gradient of the cost function for linear regression (OLS) can be written as

$$\nabla_{\theta} C(\theta_k) = \frac{2}{n} \mathbf{X}^T (\mathbf{X} \theta_k - \mathbf{y}), \quad (\text{E.4})$$

where n is the number of data points, $\mathbf{X}$ is the design matrix of our model and $\mathbf{y}$ represents our target values. For Ridge regression we have an additional penalty term for the gradient, and we will therefore express this as

$$\nabla_{\theta} C(\theta_k) = \frac{2}{n} \mathbf{X}^T (\mathbf{X} \theta - \mathbf{y}) + 2\lambda \theta, \quad (\text{E.5})$$

where λ represents the penalization.

For these gradient descent methods, we need to calculate the gradient of the cost function for every step we take. Doing this for large models is error-prone, and impractical. What we can do instead is use *Automatic Differentiation* (AD), which essentially chains together the derivative of each step as it goes, instead of making a formula for the whole thing. This method utilizes the elementary operations, such as addition and multiplication, which the computer already knows.

2. Including momentum and more advanced ways to update the learning rate

Using gradient decent has one major caveat. As the slope of the gradient decreases to a minimum, this might be the local minimum and not the global one. Additionally, as the learning rate of the method remains the same, a step that would be too big when the gradient is steep is hopelessly small along a flat one [9].

To circumvent gradient decent error-prone and computationally expensive ways, dynamic versions have been implemented. The following methods have a dynamic step sizes, where previous steps *remember* what came before them. By remember we mean their mathematical definitions make it so they retain the previous gradient $\nabla C(\theta)$ to some extent. Some also correct accordingly to its size and direction of $\nabla C(\theta)$.

For the following methods, let the gradient of the cost function be $g_t = \nabla C(\theta_{t-1})$, with $v_0 = r_0 = m_0 = 0$. For vector parameters, squares, square roots, and division are elementwise.

a. *Momentum*

$$\begin{aligned} v_{t+1} &= \beta v_t + \gamma g_t, \\ \theta_{t+1} &= \theta_t - v_{t+1}, \end{aligned} \quad (\text{E.6})$$

where v_t can be seen as the velocity or accumulator, γ is the base learning rate and β is the momentum coefficient with a typical value of 0.9, with t denoting as step of the method. In the following, gradient method parameters have the same meaning unless stated otherwise.

In the momentum gradient descent (E.6) [9], we extend the definition of the step length. In plain gradient descent we have the step length solely determined by λg_t , while in momentum this is determined by v_{t+1} . The parameter v_t can be seen as the velocity, giving the method name momentum. As the gradient g_t grows or decreases, the step size follows v_{t+1} , giving the name momentum. When the gradient is steep, the steps are large and vice versa.

Some downsides of momentum are that it can overshoot and therefore oscillate before θ_t reduces to the set tolerance, giving higher number of oscillations. As v_{t+1} depends on g_t and no previous step is forgotten, a major change in the gradient can take many iterations. The more complex the plane becomes, the more important these issues this are.

The highest learning rate for momentum is given by $\gamma_{\max} = 2(1+\beta)/\lambda_{\max}$ where $\lambda_{\max}$ is the largest eigenvalue of the hessian.

b. *AdaGrad*

$$\begin{aligned} r_{t+1,j} &= r_{t,j} + g_{t,j} \odot g_{t,j}, \\ \theta_{t+1,j} &= \theta_{t,j} - \frac{\gamma}{\sqrt{r_{t,j}} + \varepsilon} g_{t,j} \end{aligned} \quad (\text{E.7})$$

where t, j denotes that each feature has its own parameters. $r_{t+1,j}$ is the accumulator.

AdaGrad (E.7) builds on a similar adaptive step size as momentum, but tunes each parameter $\theta_{t,j}$ separately. This means that through $g_t \odot g_T$, frequent features have smaller step sized while sparse features have a bigger impact. The intent of AdaGrad exactly this, to train models that have sparse features.

A consequence of $g_t \odot g_T$ is that r_t never decreases. As a result the step size determined by $\frac{\gamma}{\sqrt{\varepsilon+r_{t,j}}}$ always decreases. Eventually it may come to a halt even if the best $\theta_{t,j}$ has not been reached. In convex problems such as the ones which we address in this project this is tolerable, but in cases like deep learning where the landscapes aren't convex, AdaGrad simply stops [9].

c. *RMSProp*

$$\begin{aligned} v_t &= \rho v_{t-1} + (1 - \rho) g_t^2, \\ \theta_{t+1} &= \theta_t - \frac{\gamma}{\sqrt{v_t} + \varepsilon} g_t \end{aligned} \quad (\text{E.8})$$

with ρ being the decay rate and ε being the stabilizer.

RMSProp (E.8) attempts to fix AdaGrad's eternal retention issue. With the term $(1 - \rho)g_t$ the gradient becomes weighted and exponentially decreasing. With

$\rho = 0.9$ this means that the current step θ_t remembers the 10 previous steps. This fixes the issue that AdaGrad has with penalizing features forever if they had big gradients early in the training process, but retains the usefulness for sparse features.

An issue with RMSProp lies in the start when $v_0 = 0$. This gives $v_1 = (1 - \rho)g_t^2$, heavily biasing the initial steps. This might make it overshoot early in the training, resulting in instability, especially when ρ is close to 1. At the same time, for sparse features when v_t is nearly 0 before a new occurrence, weighing sparse features more heavily due to these large steps.

d. *Adam*

$$\begin{aligned} m_{t+1} &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ r_{t+1} &= \beta_2 v_{t-1} + (1 - \beta_2) (g_t)^2, \end{aligned} \quad (\text{E.9})$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \hat{r}_t = \frac{r_t}{1 - \beta_2^t}$$

where m_t is the momentum term and r_t is the RMS term. β_1 is the momentum decay and β_2 is the decay applied to the RMS term, with typical values $\beta_1 = 0.9$ and $\beta_2 = 0.999$.

Adam (E.10) can be seen as a combination of momentum and RMSProp. The momentum term m_{t+1} accelerates the learning when the gradient is consistent along one direction, while the RMS term v_{t+1} dampens as size of the gradient g_t decreases.

$$\theta_{t+1} = \theta_t - \gamma \frac{\hat{m}_t}{\sqrt{\hat{r}_t} + \varepsilon} \quad (\text{E.10})$$

To account for the RMSProp behaviour that leads to biasing for initial steps, the terms $\hat{m}_t$ and $\hat{v}_t$ include a bias correction. Adam yields a combined adaptive learning method including the best features from momentum and RMSProp while also correcting for their disadvantages with (E.10) [9].

In this project these methods eqs. (E.6) to (E.8) and (E.10) are implemented using the modules Optax and JAX. Note how the meaning of the learning rate λ changes dependent on how they are implemented in each model. To showcase this, we will see how many iterations are needed to reach set tolerances.

3. Stochastic gradient descent

There are different ways to optimize a model, based on what you wish to accomplish by optimizing. When we're looking at a model that has $n = 100$ data points, we don't really think about the cost of running the program, as it won't be large. But when we instead start working with large models, such as neural networks, the cost of running the program will be significant.

Stochastic gradient descent (SGD) tackles the cost of a model, by making each step cheaper and accepts a noisy direction as the price. We will take account for both of these factors when making our SGD code, so that we don't end up with the cheapest model, but also the noisiest so that there's no accuracy in the model.

SGD works by implementing a random subset of the data, called a *mini batch*, and finding the gradient of this mini batch. We can decide the size of the mini batches M , for the n data points, which will give n/M number of mini batches. As in equation E.1, we have the same method for updating the optimal parameter for SGD, but now per mini batch. The update becomes

$$\theta_{t+1} = \theta_t - \frac{\gamma_t}{M} \nabla C(\theta_t), \quad (\text{E.11})$$

where $\nabla C(\theta_t)$ is the gradient of the cost function used, in the same way as we did for the standard gradient descent method.

If we have 10 mini batches for $n = 100$ data points, there will be 10 data points in each batch. When these mini batches have been passed over, this is called an *epoch*. We can decide how many epochs we want to run, and for each epoch that is ran, the mini batches will be randomly shuffled.

To study how sensitive SGD is to the learning rate schedule itself, rather than using a single γ , we also implement a decaying schedule given as

$$\gamma_t = \frac{t_0}{t + t_1}, \quad (\text{E.12})$$

where t is the current step number, and t_0 and t_1 together are the free parameters that control the schedules behaviour. Rather than keeping the learning rate γ fixed when training the data, the decaying schedule lets it shrink as training progresses. γ_t is largest for the very first step, and decreases toward zero as the steps t go on. This is useful because we want to take large steps at the very start, when we're far away from the minimum, but as we get closer we want to take smaller steps in order to get as close as possible. In figure 4 an illustration of how the decaying schedule looks when searching for the minimum.

The parameter t_0 decides the initial step size, and t_1 decides for how many steps the learning rate stays close to its initial value before decay becomes significant. When deciding t_1 , the total number of steps that will run need to be taken account for. If t_1 is small compared to the number of steps, γ_t will decay to a negligible value before the training is complete, and the schedule will underperform.

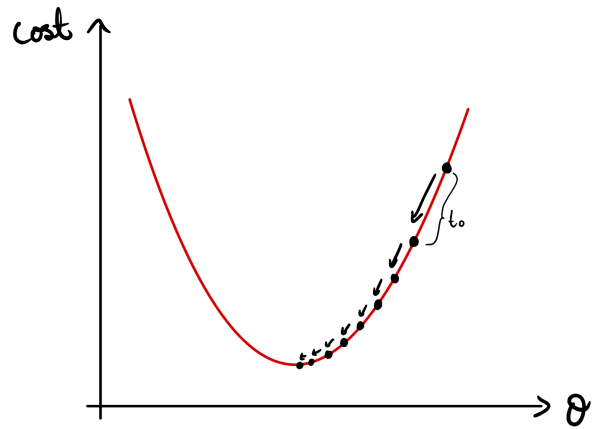


Figure 4: Illustration of the decaying schedule used for stochastic gradient descent.

III. RESULTS

A. Ordinary Least Squares (OLS) for the Runge function

For OLS we see in Figure 5 how the polynomial degrees affect the MSE and R^2 scores as well as the coefficient $\|\theta\|_2$. The noise floor is σ^2 and marked in a black dotted line at 0.01. The lowest test MSE is: 1.871×10^{-2} and the highest R^2 score is 0.8489, both at degree 8 which is marked with a grey dashed line.

We repeat the analysis at degree 8 for 15 values of n in the interval $[40, 4000]$ and 10 noise levels $\sigma \in [0.01, 1]$, this can be showed in Figure 6.

The lowest test MSE is 3.309×10^{-3} is found for $n = 4000$ and $\sigma = 0.01$. While the highest test R^2 score is 0.9562 found for $n = 322$ and $\sigma = 0.01$. These are marked in the heatmap which shows the test MSE and R^2 with 2 significant digits.

We can see how the OLS coefficients θ_j behave as a function of the degree of the polynomial in Figure 7

B. Adding Ridge regression for the Runge function

An analysis of how the MSE, R^2 scores and $\|\theta\|_2$ change over different λ values for a polynomial of degree 15 can be viewed in Figure 8. The best λ was extracted the same way as the best polynomial degree was for OLS and gave the best λ at 0.040. This gives the lowest test MSE at 2.562×10^{-2} , and the highest R^2 score at 0.7931 with effective degrees of freedom at 7.78.

The coefficients θ_j were analysed as a function of λ in order to get a scope of how the penalty parameter λ affects the magnitude of the coefficients θ_j . This can be viewed in Figure 9.

Despite $\lambda=0.040$ is the best for a polynomial of degree 15, we continue to use it forward since we want to ana-

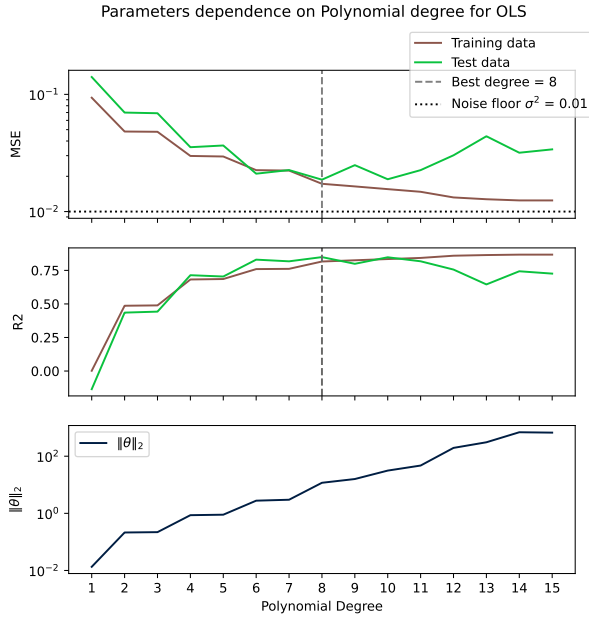


Figure 5: Test MSE is minimal and R^2 Score is maximal at degree 8. After this the test MSE increases while the training MSE decreases whereas the R^2 score does the opposite, indicating overfitting. The $\|\theta\|_2$ shows the overall magnitude of the coefficient vector as a single number per polynomial degree

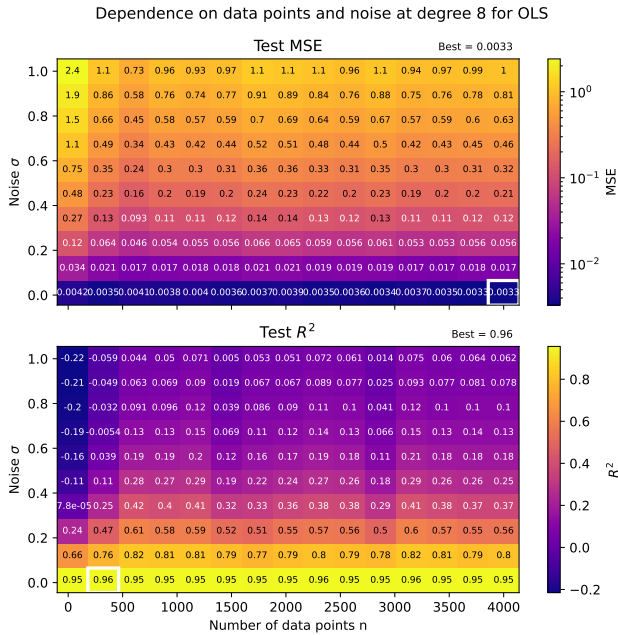


Figure 6: The lowest test MSE at 3.309×10^{-3} at $n = 4000$ and $\sigma = 0.01$. The best test R^2 score at 0.9562 at $n = 322$ and $\sigma = 0.01$

lyze how the MSE, R^2 and $\|\theta\|_2$ behave over polynomial

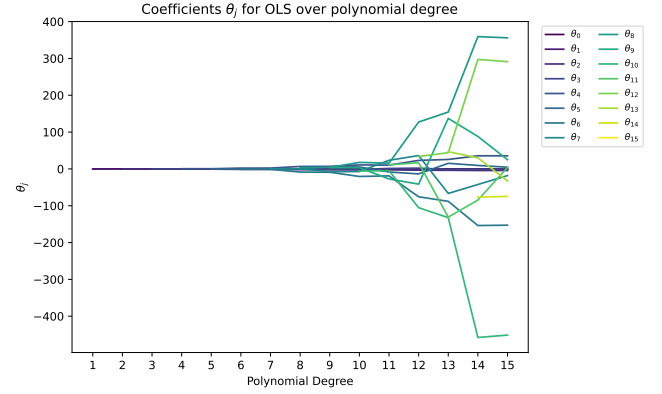


Figure 7: Coefficients stay small at a low polynomial degree and blow up at higher degrees, which is consistent with the overfitting seen in Figure 5.

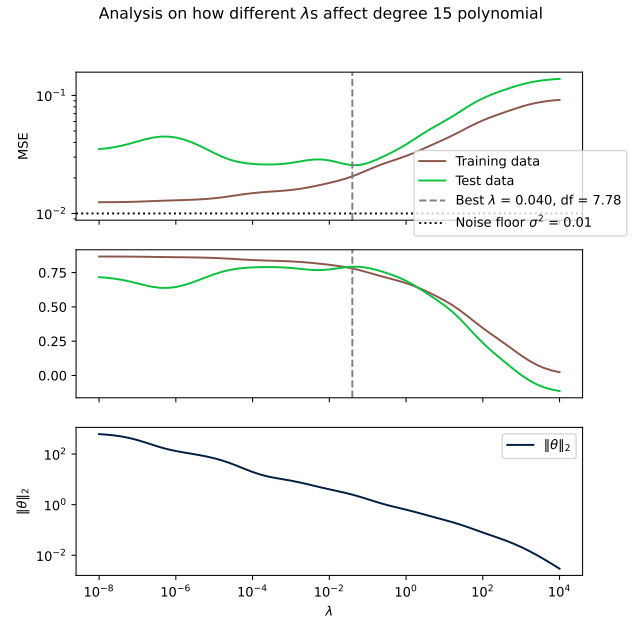


Figure 8: The best $\lambda = 0.040$ with a degree of freedom at 7.78. The $\|\theta\|_2$ shows the overall magnitude of the coefficient vector as a single number per value of λ

degree. This can be viewed in Figure 10. The lowest MSE is 2.323×10^{-2} and the highest R^2 score is 0.8125 at the best polynomial degree 12.

As done for OLS, we visualized the dependence on number of data points and noise over the same span. This was done by using the best polynomial degree and the best λ and the results are shown in Figure 11. The lowest test MSE is 2.936×10^{-3} for $n = 4000$ and $\sigma = 0.01$ and the highest R^2 score is 0.9599 for $n = 4000$ and $\sigma = 0.01$.

The coefficients θ_j as a function of polynomial degree are shown in Figure 42 in the appendix C.

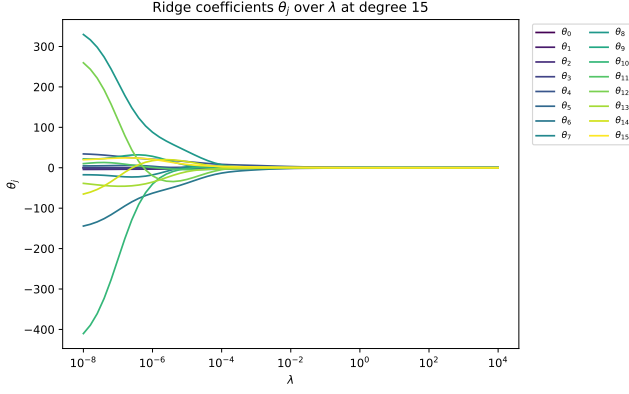


Figure 9: For smaller λ 's the solution looks like OLS (see Figure 7) with large values for the coefficients θ_j . As λ increases, these coefficients shrink.

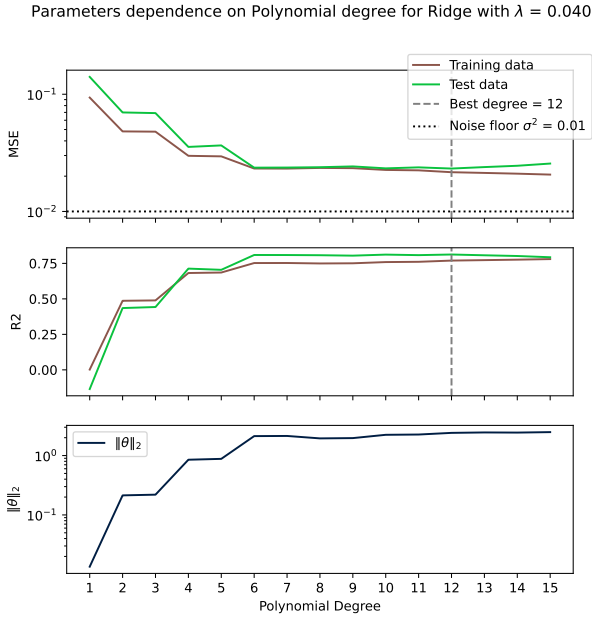


Figure 10: Test MSE is minimal and R^2 score is maximal at degree 12. The penalty parameter keeps the test MSE from blowing up and the training MSE from decreasing towards the noise floor in the same degree range as OLS seen in Figure 5

C. The bias-variance trade-off and the bootstrap

We recreated Figure 2.11 from Hastie, et al [7] in Figure 12 by extending the degree up to 20 to see the test error grow.

The bias-variance tradeoff using the bootstrap resampling technique using simpler ordinary least squares for polynomial degrees 1-15 with the number of bootstraps set to 100. This is shown in Figure 13. It marks the best polynomial degree at degree 8 for bootstrap which has

Dependence on data points and noise at degree 12 for Ridge with $\lambda = 0.040$

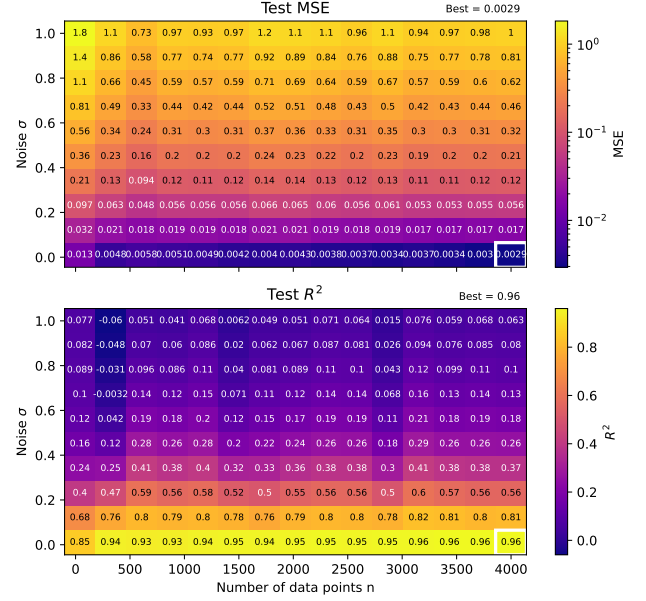


Figure 11: Lowest test MSE at 2.936×10^{-3} with $n = 4000$ and $\sigma = 0.01$. The highest R^2 score at 0.9599 at $n=4000$ and $\sigma=0.01$

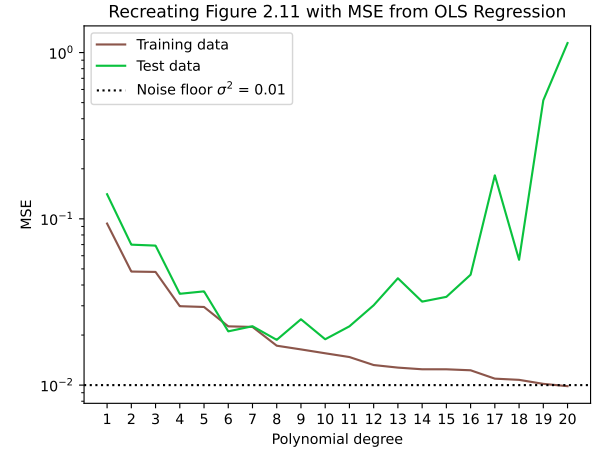


Figure 12: Recreation of Fig 2.11, Hastie et al. [7] to visualize how model complexity affects test and training error

the lowest MSE at 3.328×10^{-2} . It also plots the test MSE for OLS regression for comparison. The lowest test MSE for the OLS is 1.871×10^{-2} which is the same as values derived from Figure 5.

Three different plots with $n=40$, $n=100$ and $n=400$ were made in order to analyze how the bias-variance tradeoff changes depending on the number of data points used but still using the same number of bootstraps at 100. This can be viewed in Figure 14.

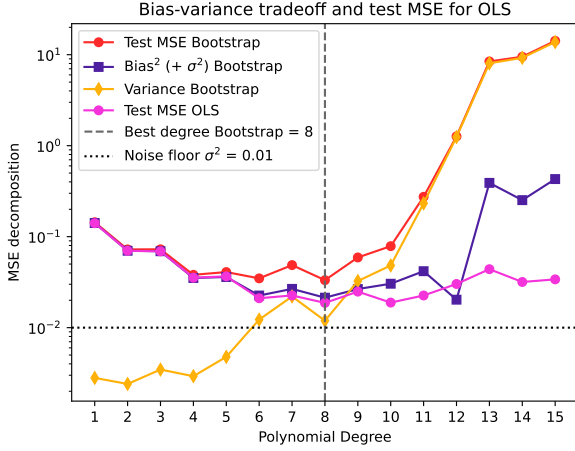


Figure 13: The best polynomial degree for simpler OLS bootstrap resampling is degree 8 where the test MSE is minimal. After this the variance shoots up and the bias has a spike after degree 12.

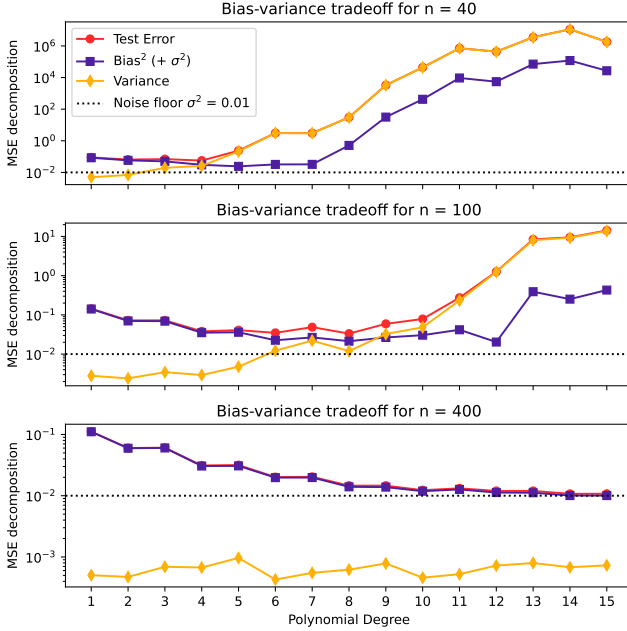


Figure 14: Using more data points (n) decreases the variance drastically so the test error will follow the bias despite the complexity of the model

D. Cross-validation as a resampling technique

In Figure 15, the MSE using the k -fold cross-validation resampling technique with $k = 5$ and $k = 10$ is compared against the MSE collected from the bootstrap resampling method. Both use simpler OLS as the linear regression method. The cross validated MSE are plotted with error bars that show the ± 1 standard deviation of the MSE across the k -folds. The cross-validated-points

for the MSE are shifted slightly from their correct degree to better visualize how $k = 5$ and $k = 10$ differ over the folds.

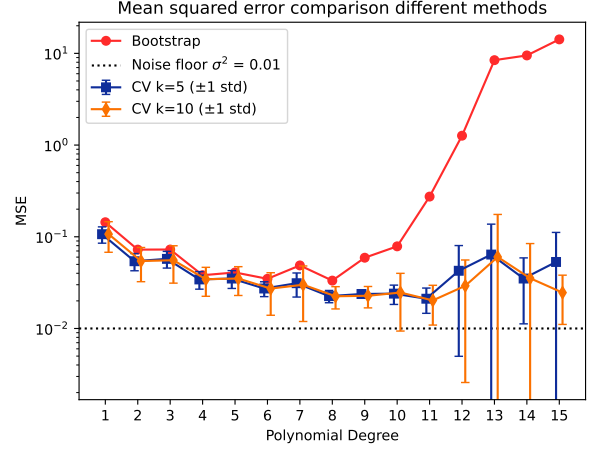


Figure 15: The three resampling methods give similar MSE scores from degree 1-8. Afterwards the bootstrap spikes and the curves for the cross-validated estimates stay almost flat until degree 11. From degree 12 the cross-validated estimates spike. Both cross-validated curves decrease again after degree 13, but cross-validated with $k = 5$ has another spike after degree 14, whereas the cross-validated with $k = 10$ continues decreasing

The subsequent best polynomial degrees and the lowest MSE derived from the data which makes up Figure 15 are displayed in Table I.

Table I: Best polynomial degree and MSE for resampling techniques: bootstrap and k -fold cross-validation using linear regression method OLS. For cross-validation the uncertainty is ± 1 std of the MSE across the k folds.

Method	Best degree	MSE ($\times 10^{-2}$)
Bootstrap	8	3.328
CV, $k = 5$	11	2.111 ± 0.65
CV, $k = 10$	11	2.024 ± 0.93

We then replace OLS by Ridge regression and study the dependence on the polynomial degree and the penalty parameter λ . We do this for k -fold cross-validation with $k = 5$ and $k = 10$ for 100 points of $\lambda \in [10^{-8}, 10^4]$. The best λ values and the lowest cross-validated MSE for each polynomial degree are shown in Table II.

The cross-validated MSE for polynomial degrees 4, 8, 12 and 15 were then plotted over the penalty parameter λ in Figure 16. $k = 5$ is marked with solid lines and $k = 10$ is marked with dashed lines. This was done in order to see how the curve for the MSE behaves over the penalty

Table II: The best cross-validated MSE is $(2.068 \pm 0.54) \times 10^{-2}$ for $k = 5$ folds and $(1.918 \pm 0.78) \times 10^{-2}$ for $k = 10$ folds over the interval $[10^{-8}, 10^4]$ for λ

Degree	Best λ		CV-MSE ($\times 10^{-2}$)	
	$k = 5$	$k = 10$	$k = 5$	$k = 10$
1	1.00×10^4	1.00×10^4	10.61 ± 2.16	10.61 ± 3.95
2	3.05	1.00	5.406 ± 1.16	5.437 ± 2.22
3	9.33	5.34	5.561 ± 1.17	5.503 ± 2.40
4	1.07×10^{-1}	2.48×10^{-1}	3.423 ± 0.75	3.431 ± 1.25
5	3.28×10^{-1}	3.28×10^{-1}	3.453 ± 0.76	3.456 ± 1.27
6	2.01×10^{-2}	2.66×10^{-2}	2.681 ± 0.43	2.668 ± 1.15
7	4.64×10^{-2}	4.64×10^{-2}	2.764 ± 0.45	2.734 ± 1.17
8	5.34×10^{-4}	4.04×10^{-4}	2.240 ± 0.36	2.240 ± 0.63
9	7.06×10^{-4}	5.34×10^{-4}	2.315 ± 0.30	2.228 ± 0.61
10	1.00×10^{-4}	1.75×10^{-4}	2.224 ± 0.27	2.185 ± 0.61
11	1.87×10^{-5}	5.72×10^{-5}	2.096 ± 0.49	1.981 ± 0.71
12	1.87×10^{-5}	1.07×10^{-5}	2.068 ± 0.54	1.918 ± 0.78
13	5.72×10^{-5}	2.48×10^{-5}	2.093 ± 0.59	1.943 ± 0.86
14	1.75×10^{-4}	1.00×10^{-4}	2.130 ± 0.49	1.972 ± 0.78
15	3.05×10^{-4}	2.31×10^{-4}	2.180 ± 0.50	2.009 ± 0.81

parameter λ and the degrees chosen as evenly spaced as possible.

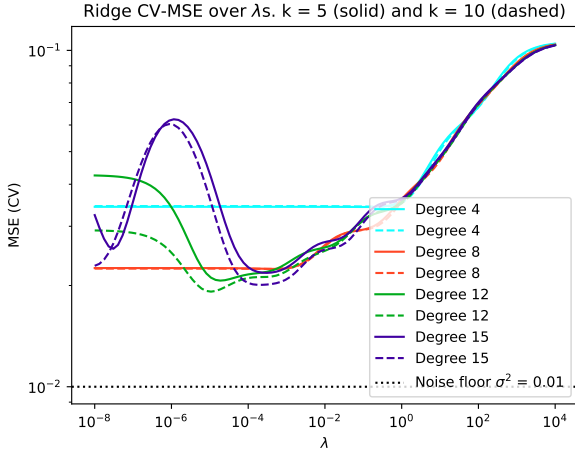


Figure 16: For low degrees the cross-validated MSE needs little to no regularisation and the curve is flat for low λ 's. When λ increases all degrees converge to the same value

E. Gradient descent code, with analytical gradients and automatic differentiation

For OLS and Ridge we find the difference between the optimal parameters θ , using the analytical expression and automatic differentiation. This confirms that our analytical gradient formula is correct. In table III, the gradient differences are shown for both methods.

	$ \theta_{\text{AD}} - \theta_{\text{analytical}} $
OLS	8.33×10^{-16}
Ridge	8.33×10^{-16}

Table III: Comparison of gradients (AD vs. analytical) for OLS and Ridge regression.

We want to find the best learning rate to use for our gradient descent. In figure 17, the solution of the gradient descent compared to the closed form solution is plotted as a function of the learning rate γ . The learning rate limit γ_{max} is plotted for both OLS and Ridge, and we find that the limits are for OLS: $\gamma_{\text{max}} = 0.349$, and for Ridge: $\gamma_{\text{max}} = 0.348$.

To find the best learning rate, we plot the number of iterations required for the different learning rates, for the gradient descent solutions. In figure 18, we find the best learning rate γ_{best} by finding the minimal number of iterations required, and we get $\gamma_{\text{best}} = 0.344$ for both methods.

We can find the difference between the analytical solution and the closed form solution, which tells us how close the gradient descent's final iterate gets to the optimal value. We do this for both γ_{max} and γ_{best} . The results for these are found in table IV. In this table, the number of iterations it takes to find the optimal value is also shown. Note that for γ_{max} , the number of iterations it takes is 10 000, but this is also the number of iterations the program is set to. So it doesn't reach the optimal value within the set number of iteration, but rather the code has just reached its limit.

We can further study the number of iterations required to reach the optimal value for different values of the learn-

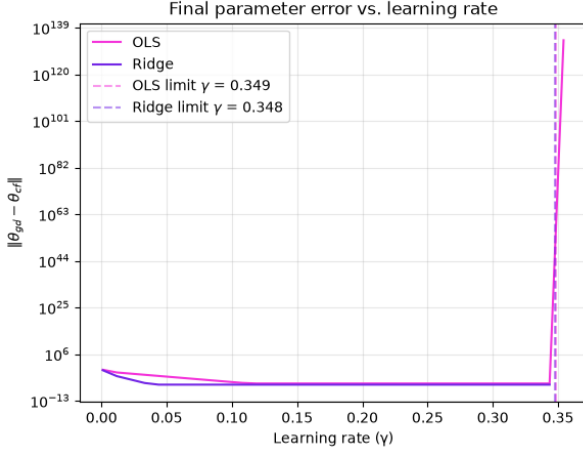


Figure 17: The difference between the optimal parameter for the gradient descent solution and closed form solution as a function of the learning rate, for OLS and Ridge. The limit for the learning rate $\gamma_{\max} = 0.349$ for OLS and $\gamma_{\max} = 0.348$ for ridge.

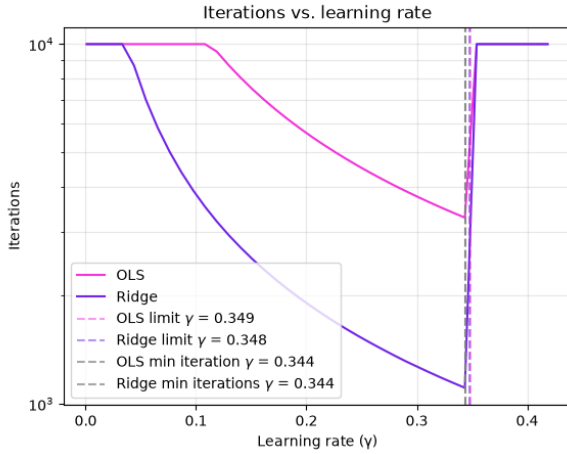


Figure 18: Iterations as a function of the learning rate for OLS and Ridge. The learning rate that requires the least number of iterations is $\gamma_{\text{best}} = 0.344$ for both methods.

ing rate, and in figure 19 we do exactly this.

When we've now found the best value for the learning rate, we can use this to study the cost function of the different methods. In figure 20, we've done this for both $\gamma_{\max}$ and γ_{best} .

We've now trained the gradient descent method on the data we set aside, but how well does it hold up when tested? In figure 21, the MSE of the test data is compared to the training data.

For Ridge, where there is a penalty λ , and we can study how this penalty effects the MSE of the training data and compare to the MSE of the test data. This has been

	$ \theta_{\text{gd}} - \theta_{\text{closed form}} $		Iterations to converge	
	$\gamma_{\max}$	γ_{best}	$\gamma_{\max}$	γ_{best}
OLS	1.858×10^{-2}	7.718×10^{-7}	10 000	3293
Ridge	1.852×10^{-2}	2.581×10^{-7}	10 000	1108

Table IV: Comparison of gradient descent for the closed form and analytical solution, for the learning rate limit $\gamma_{\max}$ and the best learning rate γ_{best} .

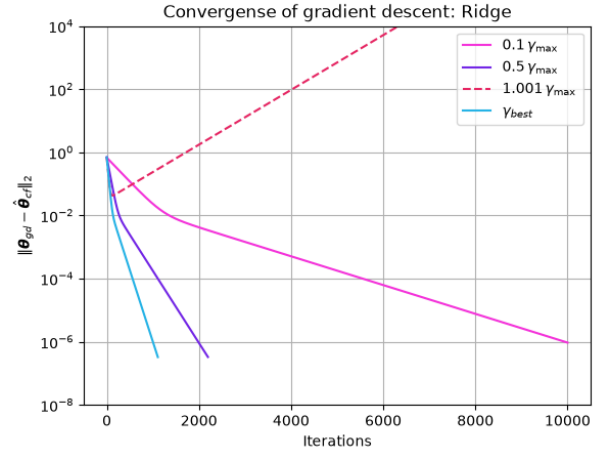
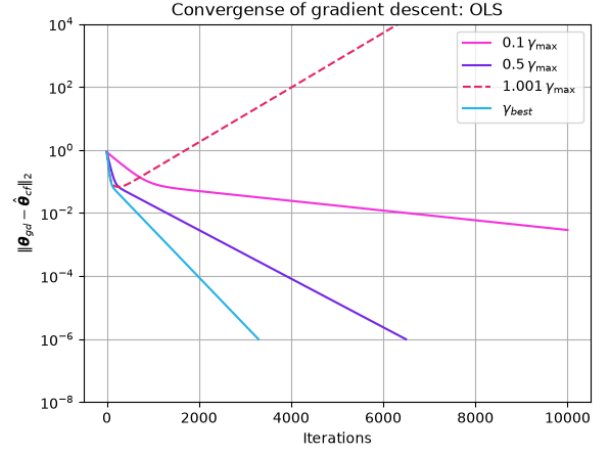


Figure 19: Convergence of the gradient descent method for OLS and Ridge, for different values of the learning rate γ . For a learning rate larger than $\gamma_{\max}$, the solution diverges.

plotted in figure 22.

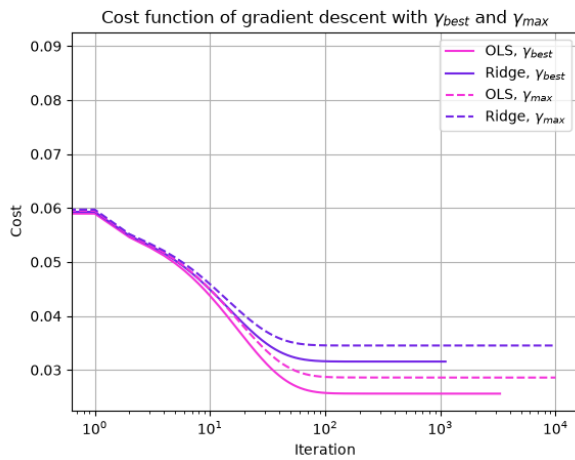


Figure 20: Cost function of OLS and Ridge as a function of number of iterations. The solution with the best learning rate converges faster. Values of the final cost and iterations is shown in table IV.

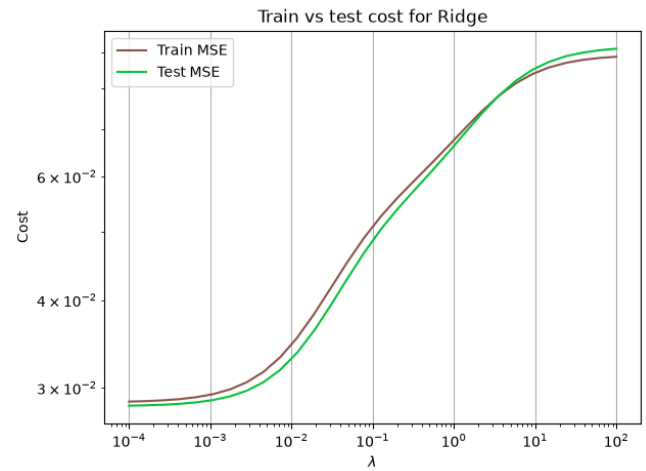


Figure 22: The MSE of the training and test data as a function of the penalty factor λ , for Ridge.

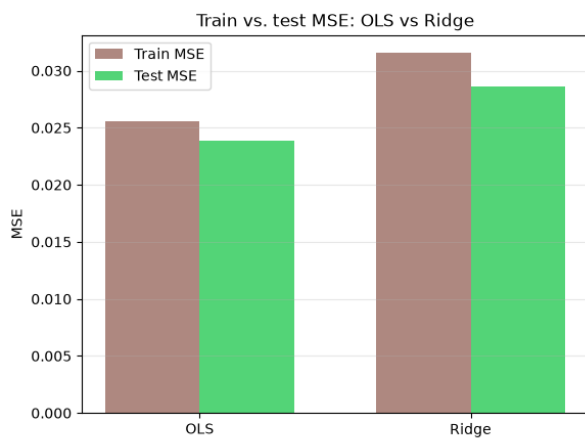


Figure 21: Comparing the MSE of the test and training data for OLS and Ridge.

F. Including momentum and more advanced ways to update the learning rate

The gradient descent methods Plain Gradient Descent (PGD), Momentum, AdaGrad, RMSProp and Adam have a varied number of needed iterations for convergence to have excess cost be reduced below the tolerance. In fig. 23 we see that for a tolerance of 10^{-4} all models eventually converge for some learning rates. The number of iterations needed to so change drastically, but trend towards less needed iterations as learning rate and hence the step size increases. Neither AdaGrad nor PGD manage to converge for learning rates close to $2/\lambda_{\max}$. Momentum fails to converge above $2(1 + \beta)/\lambda_{\max}$, which is seen as the limit for the method’s convergence.

When we decrease the tolerance to 10^{-8} , the possible learning rates for convergence decreases drastically. RMSProp fails to converge for any rate, and plain gradient descent has a severely limited learning rate range. We see that for large rates, Momentum converges the fastest, but over all Adam does reasonably well for all rates, converging at only 200 iterations more than Momentum does at its best. For learning rates above momentum’s limit, it does not converge, but both PGD and Adam does, although PGD’s $\gamma_{\max}$ should be surpassed.

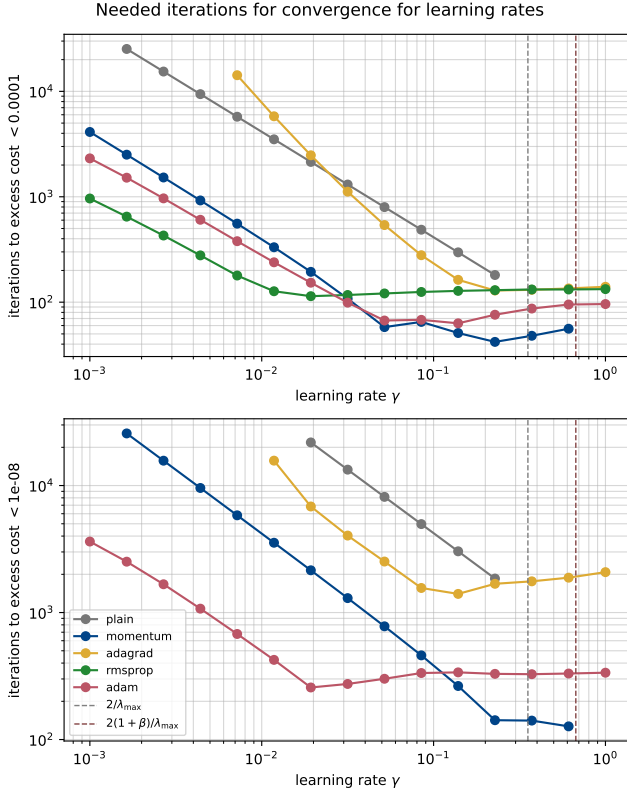


Figure 23: The various models have different ranges of learning rate γ that allows the cost version to converge.

G. Lasso regression

Before comparing the different Lasso solvers, we first check why `lasso_gd` uses an explicit sub gradient rule rather than letting JAX differentiate the ℓ_1 penalty automatically. The ℓ_1 term $|\theta|$ is not differentiable at $\theta = 0$, so its true derivative there is only defined as a sub gradient, any value in $[-1, 1]$. Table V shows what JAX’s automatic differentiation (AD) returns at $\theta = 0$ and at nearby points.

θ	$d \theta /d\theta$ (JAX AD)
-1.0×10^{-3}	-1.0
-1.0×10^{-9}	-1.0
0.0	+1.0
1.0×10^{-9}	+1.0
1.0×10^{-3}	+1.0

Table V: Automatic differentiation (JAX) derivative of $|\theta|$ at $\theta = 0$ and nearby points. At $\theta = 0$, JAX returns +1.0 rather than the true sub gradient interval $[-1, 1]$ or a value consistent with the sign of nearby points — an artefact of how `jnp.abs` is implemented, not a principled sub gradient choice.

We’ve now also implemented the method Lasso regression and will compare to OLS and Ridge. Since Lasso doesn’t have a closed form solution for us to compare to, we’ve instead compared different way of doing a Lasso regression to each other.

In table VI we compare the SciKit Learn package [10] to the gradient descent and coordinate descent methods, and we look at the number of iterations it takes to reach the optimal values.

Method	Iterations to converge $ \theta - \theta_{\text{sklearn}} $	
Scikit-learn	307	—
Gradient descent	513	2.46×10^{-7}
Coordinate descent	181	1.06×10^{-7}

Table VI: Comparison of Lasso solvers: number of iterations to converge, and maximum absolute deviation from the scikit-learn reference solution.

We now need to compare Lasso to OLS and Ridge, in figure 24 we have compared the cost functions of OLS, Ridge and Lasso for both gradient and coordinate descent.

We also need to see how Lasso regression holds up when we test it on the data the model hasn’t been trained on. We extend the OLS/Ridge comparison of figure 21 by adding Lasso, shown in figure 25

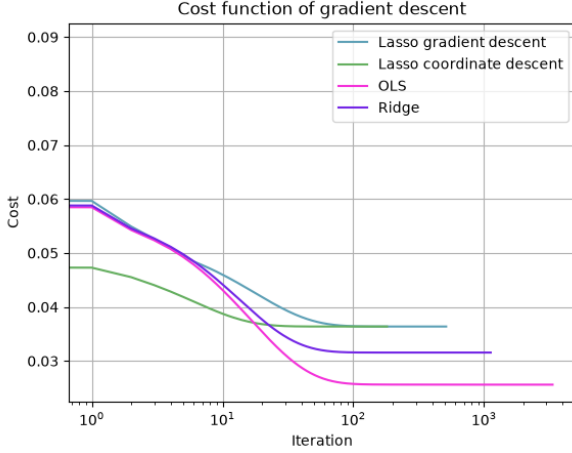


Figure 24: Cost function for OLS, Ridge and Lasso for Gradient descent and Coordinate descent. Values for iterations and final cost for the two Lasso methods is found in table VI.

Method	Train	Test
OLS	2.561×10^{-2}	2.390×10^{-2}
Ridge	2.677×10^{-2}	2.382×10^{-2}
Lasso	2.688×10^{-2}	2.381×10^{-2}

Table VII: Train and test MSE for OLS, Ridge and Lasso

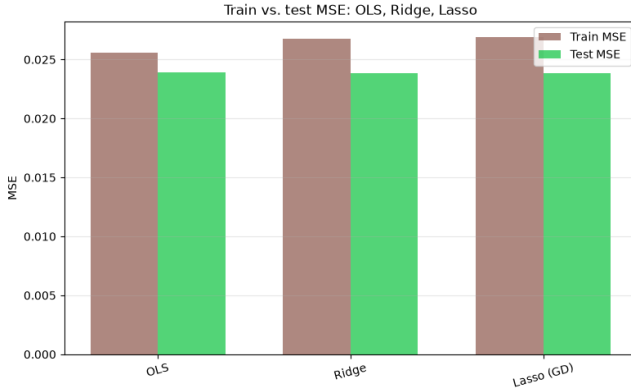


Figure 25: Comparing the MSE of the test and training data for OLS, Ridge and Lasso. Values for different MSE are found in table VII.

Since we also have a penalty factor λ for Lasso regression, we can again look at how λ effects the MSE for the training and test data. In figure 26, the MSE for Lasso and Ridge is plotted to compare how well the model works on the test data.

Figure 27 shows how the individual coefficients of Ridge and Lasso shrink as λ increases. For Ridge, all coefficients shrink smoothly and continuously toward zero as

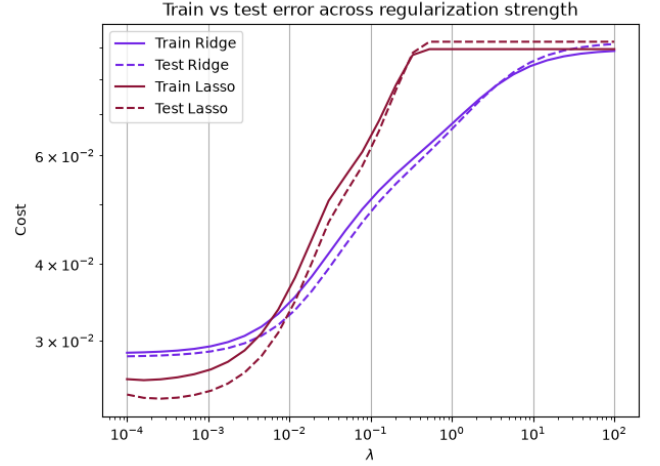


Figure 26: The MSE of the training and test data as a function of the penalty factor λ , for Ridge and Lasso.

λ grows, but none of them reach exactly zero. Lasso instead drives coefficients to exactly zero one at a time as λ increases, performing implicit feature selection rather than just shrinkage. This matches what we see at $\lambda = 10^{-2}$ specifically: scikit-learn’s Lasso solution is $\theta = [0, -0.569, 0, 0.381, 0]$, with three of the five coefficients set exactly to zero, while the corresponding Ridge solution at the same λ has no zero entries at all, with the solution being $\theta = [-0.005, -0.574, 0.015, 0.387, -0.012]$.

H. Stochastic gradient descent

For the Stochastic Gradient Descent (SGD) method, there are multiple parameters we can change, and thereafter study the effect of. Firstly we study what happens when the batch size is changed, but the number of batches is held constant. In figure 28, this is plotted for three different batch sizes. In figure 29 we have plotted a heat map of the varying batch size in order to show how the cost is effected for a range of batch sizes.

Next, in figure 30, we change the learning rate and plot the cost function for these different learning rates. In figure 31, we have plotted a heat map of the varying learning rate.

In figure 32 we use the decaying schedule, given in equation E.12, to show the cost function for OLS. We then compare the schedule directly against holding γ constant at the same initial values, shown in figure 35. Since the two cost functions are relatively similar, we have also shown the difference between the in this figure, as well as the actual learning rate that is used for the two methods. With t_1 chosen relative to the total number of iteration the program performs, the schedule

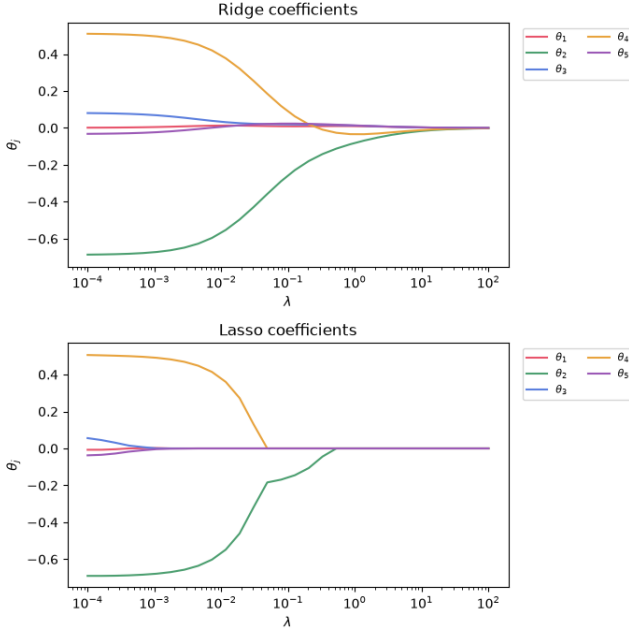


Figure 27: The coefficients for Ridge and Lasso as a function of penalty λ . For smaller penalty values, the coefficients are large, and for ridge they shrink toward zero as the penalty increases. For Lasso they are put to exactly zero one after the other.

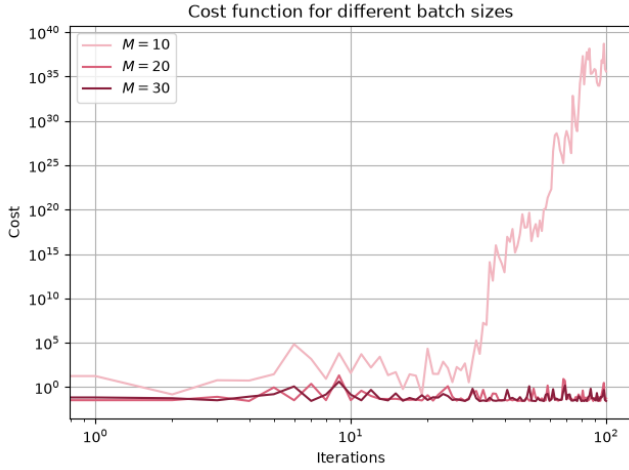


Figure 28: Cost function for SGD for different batch sizes M . For $M = 10$, the cost increases by a lot more than for larger batch sizes.

reaches within 1 % the closed form solution in the same number of iterations as when the learning rate is held constant: 480 iterations. The parameters ended with $|\theta_{\text{schedule}} - \theta_{\text{constant}}| = 1.28 \times 10^{-2}$, for the two methods. We compare plain SGD with the four adaptive optimiser: momentum, Adagrad, RMSprop and Adam, for both OLS and Ridge, as shown in figure 36. Table VIII reports, for each method, the number of iterations

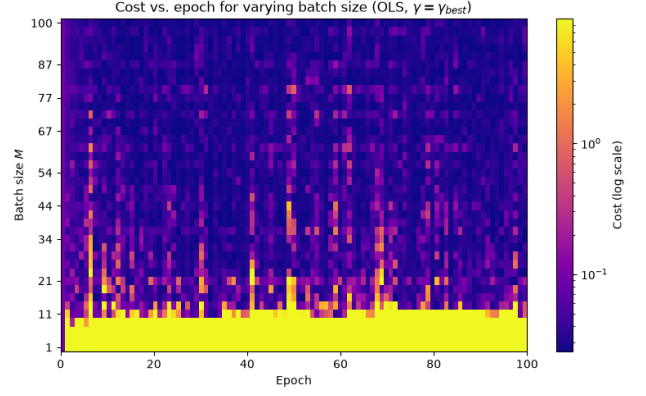


Figure 29: Cost function for SGD for different batch sizes M .

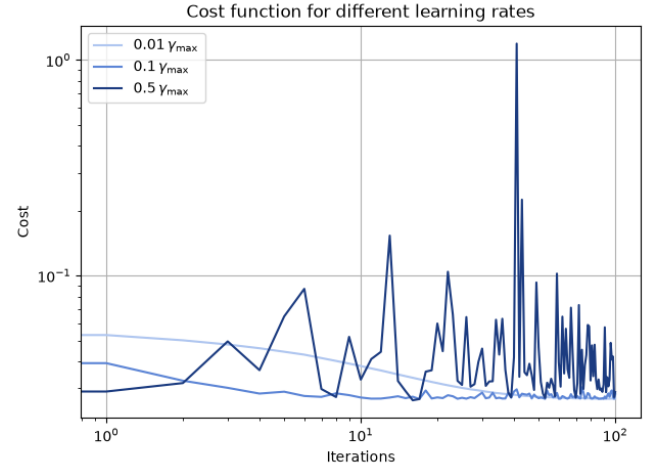


Figure 30: Cost function for SGD for different learning rates γ . For a learning rate of $0.5 \gamma_{\text{max}}$, cost starts to become unstable.

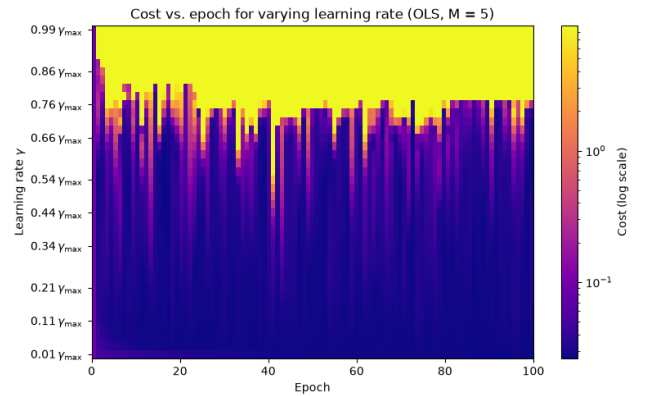


Figure 31: Cost function for SGD for different learning rates γ .

needed to reach within 1% of the closed-form cost, and

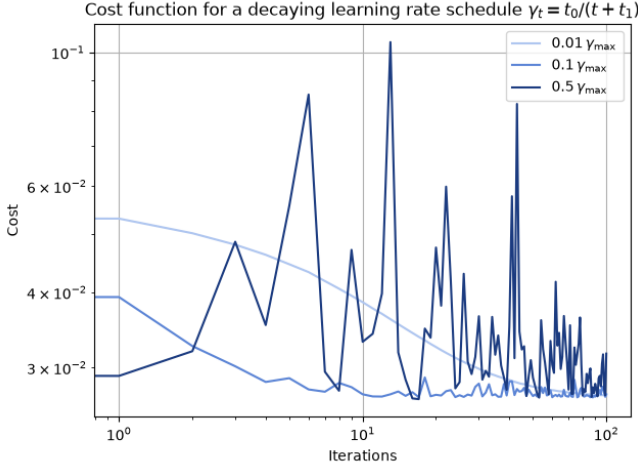


Figure 32: Cost function for SGD with a decaying learning rate schedule $\gamma_t = t_0/(t + t_1)$, for three initial rates matching those of figure 30. Cost here is generally lower than in figure 30.

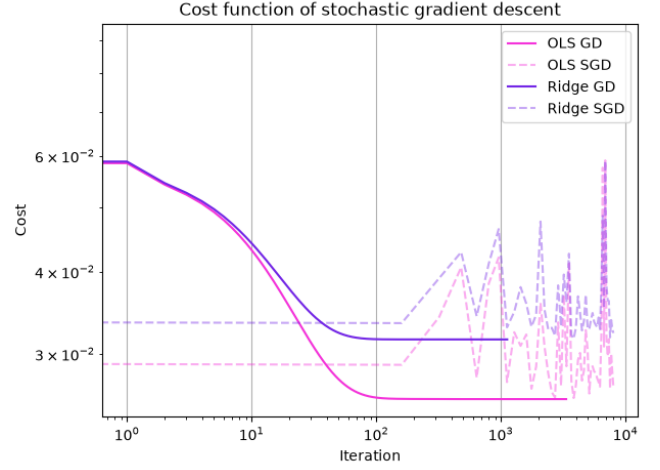


Figure 33: Cost function for OLS and Ridge using gradient descent and stochastic gradient descent. Difference between final values of cost and iterations to converge are shown in table IX.

the final distance to the closed-form solution.

Optimiser	γ	OLS		Ridge	
		Iter.	$ \theta - \theta_{cf} $	Iter.	$ \theta - \theta_{cf} $
Plain SGD	0.1	2720	3.71×10^{-2}	3840	2.87×10^{-2}
Momentum	0.01	6240	2.39×10^{-2}	1120	2.29×10^{-2}
Adagrad	0.1	1760	2.71×10^{-2}	1120	1.52×10^{-3}
RMSprop	0.1	6240	1.95×10^{-1}	n/a	1.54×10^{-1}
Adam	0.1	4640	6.06×10^{-2}	n/a	8.94×10^{-2}

Table VIII: Comparison of SGD optimisers for OLS and Ridge: iterations needed to reach within 1% of the closed-form cost (out of 8000 total updates run), and the final distance to the closed-form solution.

We can also compare SGD to standard gradient descent, and see how they compare to each other. In figure 33, we compare the two models, and table IX reports the number of iterations each took to converge and the resulting distance between their solutions.

Method	GD iter.	SGD iter.	$ \theta_{SGD} - \theta_{GD} $
OLS	3341	8001	3.71×10^{-2}
Ridge	1124	8001	2.87×10^{-2}

Table IX: Iterations to converge for gradient descent (GD) and stochastic gradient descent (SGD), for OLS and Ridge regression.

We also need to study how this model holds up when tested, just as we did for gradient descent. In figure 34, the MSE for the training and test data is shown for OLS

and Ridge, for both GD and SGD, with the exact values reported in table X.

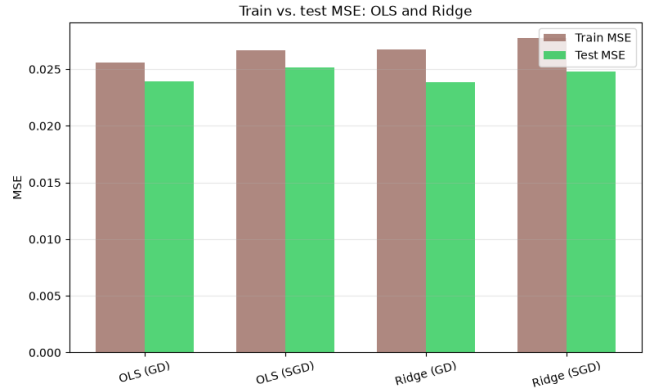


Figure 34: Train and test MSE for OLS and Ridge, via gradient descent (GD) and stochastic gradient descent (SGD). Values for the different MSE are found in table X.

Method	Train MSE	Test MSE	Gap (test - train)
OLS (GD)	2.5608×10^{-2}	2.3899×10^{-2}	-1.7087×10^{-3}
OLS (SGD)	2.6703×10^{-2}	2.5164×10^{-2}	-1.5394×10^{-3}
Ridge (GD)	2.6765×10^{-2}	2.3821×10^{-2}	-2.9446×10^{-3}
Ridge (SGD)	2.7738×10^{-2}	2.4792×10^{-2}	-2.9462×10^{-3}

Table X: Train and test MSE for OLS and Ridge, via gradient descent (GD) and stochastic gradient descent (SGD).

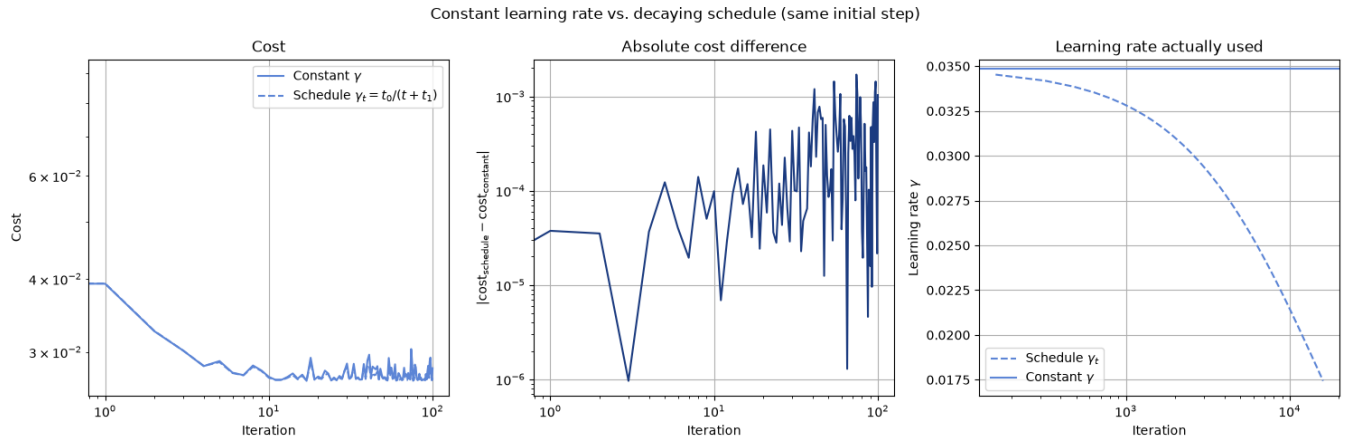


Figure 35: Left: cost function for SGD with a constant learning rate versus a decaying schedule with the same initial step, for OLS. Middle: absolute difference between the two cost curves. Right: the learning rate each method actually used at each iteration.

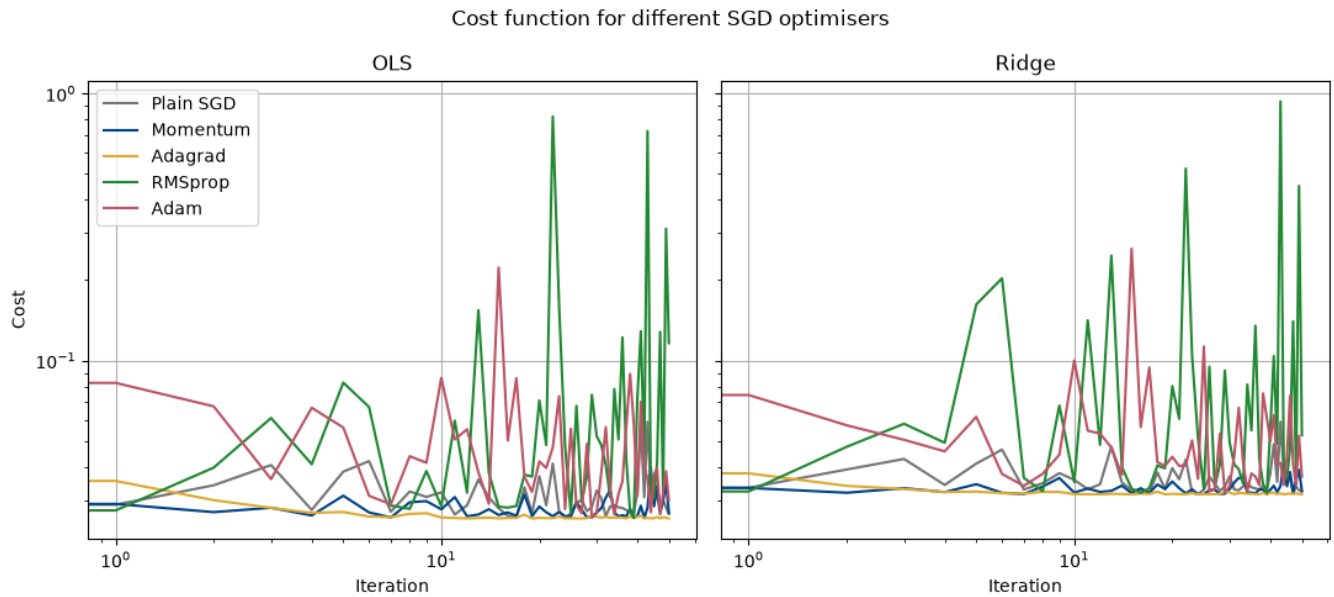


Figure 36: Cost function for SGD with plain, momentum, Adagrad, RMSprop and Adam optimisers, for OLS (on the left) and Ridge (on the right). Cost is generally higher for Ridge.

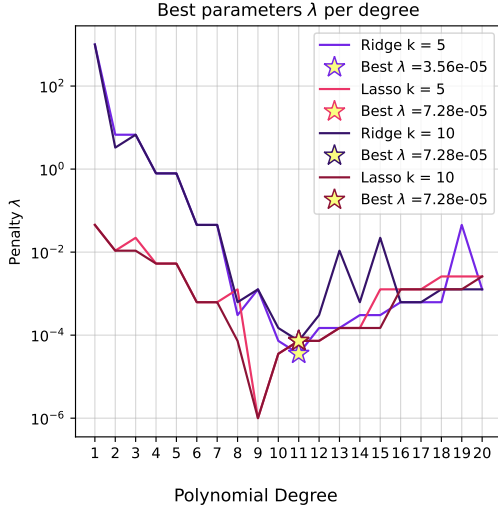


Figure 37: The penalty λ is shown for each degree per by their minimised cost functions as in fig. 38. The λ varies for the models, but are all on the same order of magnitude for the degree and chosen.

I. Cross-validation with hyperparameters

With the k-fold cross-validation method implemented as in IID 2b, we see how the MSE changes for the degrees fig. 38. For the Ridge and Lasso estimators, the best scores range between 10^{-1} and 10^{-2} for both folds 5 and 10. Their MSEs vary somewhat from degree to degree, but remain mostly stable. Note that the Ridge test and train MSEs are the closest for their respective best scores.

The OLS with 5 and 10 folds tells a different story, with its only tunable parameter being number of degrees. At best, the MSE is 2.5×10^{-2} at the 8th degree, but blows up the order of 10^2 for higher polynomials.

The best tuned models have different best penalties λ as shown infig. 37. Ridge penalties for both fold follow the same trend, and the same goes for Lasso. This is expected behaviour as per their mathematical descriptions. In both cases, the best λ follow a rough valley shape.

The best estimators were attempted to be decomposed following section IID 1, but with data limited to the training set. The result is a poor decomposition for higher degrees(fig. 39), where the error, which is the bootstrap MSE, in no way reflects the training MSE. We This is especially clear for the Ridge MSE decomposition. As for below 7 degrees we manage to decompose the MSE reasonably well with both resampling strategies, though the variance of the bootstrap and k-fold resampling can differ by a factor of 5. While size might differ, they yield a similar slopes. In both resampling methods for all folds and estimators, the bias is the most significant contributor. In the appendix in section C there is a MSE decomposition for 5 fold Ridge with the whole dataset compared to the train dataset only. When the full dataset is used, the error is similar to the test and

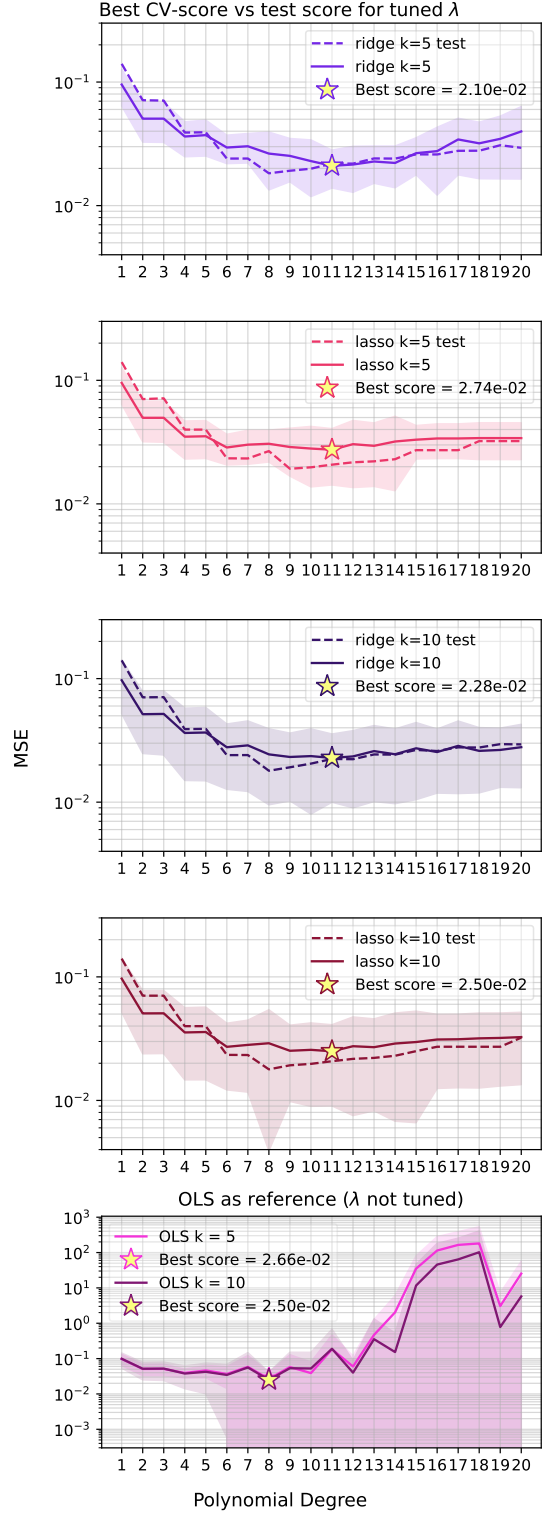


Figure 38: As a combination of both degree and penalty λ , the minimised cost function per best combination is shown. Standard deviation for the train MSEs are included. See fig. 37 for penalty per degree. F degrees 6 and onwards for Ridge and Lasso the MSE remains on the same order of magnitude. Note that OLS is only used as a reference. For all fold sizes and estimators Ridge and Lambda the best score occurs at the 11th degree.

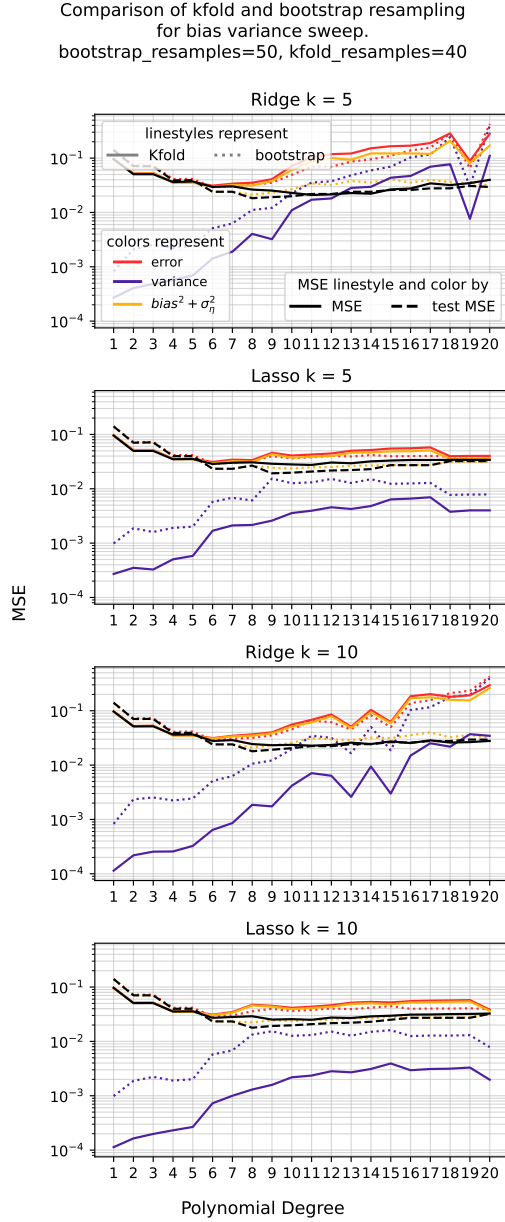


Figure 39: Bootstrap and kfold resampling to decompose MSE. The error and MSE has the same mathematical description (A.2), but uses the resampled data

train MSEs. In all cases the bias contributes the most to the error, but variance plays a bigger part as the degrees increase.

The with the best estimators for each fold refit to the whole training set, we get fig. 40. We see that all five models deviate from the Runge function, especially when the generated data is sparse. The 5- and 10-fold Lasso model does somewhat better, with the fit in the range of $x = -1.00$ and $x = -0.50$ much closer to the Runge function as opposed to Ridge models.

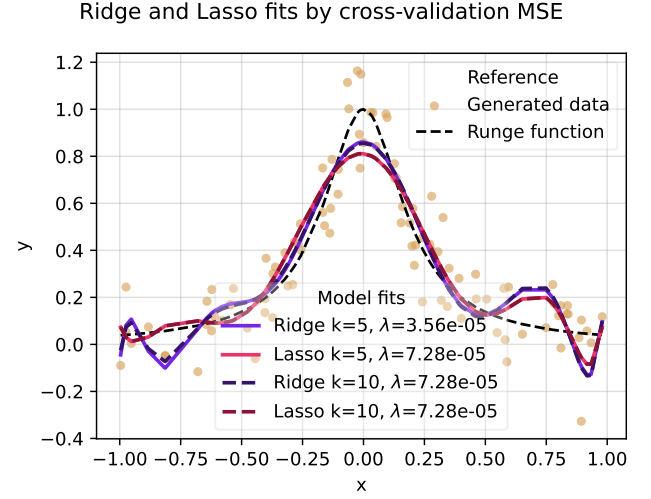


Figure 40: The fits of the estimators Ridge and Lasso with $k=5, 10$ behave similarly, but deviate from the true Runge-function and generated data from it.

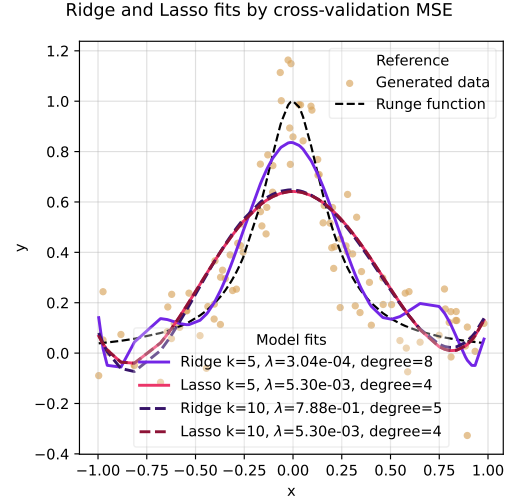


Figure 41: The cheapest fits with MSE within one standard deviation are chosen, showing how r

When we choose the cheapest permissible models instead as in fig. 41, we see that both Lasso models and the 5-fold Ridge model with their lower degrees (4 and 5) follow the curves worse than the 8 degree 5-fold Ridge fit.

IV. DISCUSSION

A. Ordinary Least Squares (OLS) for the Runge function

We start by evaluating the MSE, R^2 score and $\|\theta\|_2$ as a function of the polynomial degree. We know that by increasing the degree of the polynomial, the complexity of the model increases as well. From Figure 5 we see that the training MSE decreases towards the noise floor $\sigma^2 = 0.01$. The test MSE decreases until it reaches a minimum at degree 8 at 1.871×10^{-2} , afterwards it will fluctuate. The combination of training MSE that continues to improve whereas the test MSE worsens, shows that the model overfits at more complex degrees as discussed in the bias-variance tradeoff.

We see that the R^2 score increases when the MSE decreases. This follows from $R^2 = 1 - \text{MSE}/\text{Var}[\mathbf{y}]$, since $\text{Var}[\mathbf{y}]$ is a fixed number for the data set, decreasing the MSE will decrease the second term of the equation which increases the R^2 score. So a lower MSE will always give a higher R^2 . We can thus see that the graphs for MSE and R^2 score mirror one another as where the MSE sinks the R^2 score will increase. And the lowest MSE gives the highest R^2 score at 0.8489 at degree 8. The $\|\theta\|_2$ blow up with a magnitude of 10^4 over the 15 degrees. This is consistent with the features becoming almost linearly dependent for high degree polynomials. As a result of this small changes in the data give very large changes in θ . This can also be viewed in Figure 7 where we see that that the coefficients θ_j for OLS blow up for larger polynomial degrees at opposite signs so they cancel out one another. This is another sign of overfitting, as the coefficients of θ become huge in order to decrease the training MSE.

From the heatmaps in Figure 6 we see that the test MSE mostly follows the noise since increasing the number of data points still keeps the MSE within the same range. This holds except for very few data points (the first column). From this we can see that the noise is what impacts the test MSE the most. We see in the lowest row that the MSE test scores are still above the noise floor at $\sigma^2 = 10^{-4}$ so the degree 8 polynomial is dominated by bias.

B. Adding Ridge regression for the Runge function

Analysing a degree 15 at different penalty parameters λ shows us that very small λ 's will give almost the same result as OLS. Since a large λ corresponds to a less flexible model we see that if Figure 8 was read from right to left, it would resemble a smoother version of Figure 5. We see that the MSE has a minimum at $\lambda \approx 0.040$ with effective degrees of freedom ≈ 7.78 . This is roughly equivalent to the best polynomial degree for OLS at degree 8. Large λ 's will make the MSE rise and the R^2 score decrease towards zero.

We see that as λ increases, the Ridge coefficients decrease towards zero. We see that the large coefficients shrink together and that some will change sign or grow before going to zero. This is because Ridge will shrink the direction given by the SVD instead of shrinking each coefficient individually (Eq. A.8).

We see that by adding a penalty parameter λ the MSE, R^2 and $\|\theta\|_2$ graphs will look different over the same polynomial degrees compared to OLS. The added penalty parameter makes it so the test MSE no longer increases at high polynomial degrees, instead it stays almost constant from degree 6-15. The lowest MSE value at 2.323×10^{-2} at degree 12 is slight higher than the lowest OLS MSE at 1.871×10^{-2} at degree 8. And the R^2 score is also lower at 0.8125 compared to the OLS at 0.8489. For this particular data set we see that Ridge does not give a better prediction than OLS, but it is more stable. Ridge not returning a better MSE value than OLS may be caused by the fact that $\lambda = 0.040$ was derived from a polynomial of degree 15, so this could be too small or too large of a dampening for a degree 12 polynomial.

We also see that $\|\theta\|_2$ will only increase by two magnitudes from $\approx 10^{-1}$ to 10^0 instead of four magnitudes which happens in OLS. We can also see how these coefficients differ by comparing Figure 7 and Figure 42. We see that the magnitudes of the coefficients differ. For OLS the coefficients blow up towards ± 400 's whereas for Ridge with a penalty parameter of $\lambda = 0.040$ we get a range between -1.0 to 1.5 which shows how Ridge suppresses the parameters, thus improves the stability at more complex models.

From the heatmap we see that we get the same pattern as we did for OLS. Except that the best values for test MSE are lower at 2.936×10^{-3} compared to OLS at 3.309×10^{-3} and the R^2 score is higher at 0.9599 compared to OLS at 0.9562, but this improvement is very small. For low noise, the model is dominated by bias which decreases as the complexity of the model increases. Since Ridge remains stable at higher degrees, we can use a degree 12 polynomial instead of a degree 8 as we did in the heatmap for OLS. This reduces the bias and is why we get a slightly lower test MSE.

C. The bias-variance trade-off and the bootstrap

By increasing the polynomial degree range we see a very clear example of the Hastie et al. Fig 2.11 (Figure 1) from our Figure 12. Here we see that as the complexity increases, the training error decreases while the test error grows rapidly. We also see that at degree 19/20 the training data sinks beneath the noise floor which shows that the model is fitting the noise.

We see that for low degrees (1-5), the model is bias dominated and the variance is very low. The degree that minimises the sum of the bias and the variance is degree 8, which is the same as for OLS. After degree 8 the variance shoots up and the test MSE for the bootstrap

will follow this curve. The bias will slowly increase until degree 12 where it has a spike at degree 13. In theory, this should not happen since increasing the degree of the polynomial makes the model more flexible and it can thus follow the true function more closely. So the spike at degree 13 is a consequence of how bias is estimated. The expected prediction is derived by taking the average of 100 bootstraps, and at high degrees these are very unstable, therefore some extreme fits can derail the average. The bootstrap also has fewer unique points (63.2% of the training data) compared to the OLS where all the training data are unique points which makes the bootstrap at high-degree polynomials even more unstable.

This is supported by Figure 14. Here we see that the variance decreases rapidly when the number of data points increases. The bootstrap has then more data points to choose from and the average will be more stable. We see that theory holds for this, as the bias sinks for higher degree polynomials when there is enough data points. Although it has not been marked in the plot, we see that the more data points we have, the more the optimal degree will move to the right.

Each bootstrap sample is drawn with its own seed (i+1) in order to make the result reproducible. If we were to draw each sample with the same seed, for example 2026, each sample would contain the same data, so all the models would be the same and all predictions would match giving no variance. What seed is used will affect how the bias-variance tradeoff will look. We repeated this for different seeds (Figures 43, 44 and 45) and notice that a change in the seed will change the curve and the best degree will move. We can therefore not conclude that the best degree is 8 since this is dependent on what seed is used when x and y are resampled. But, we can say it must lie somewhere between 4 -8, since this is where the model is the most stable for all the seeds.

D. k -fold Cross-validation as a resampling technique

By setting $k = 5$, we get 5 folds where each fold is about 20% of the data set, whereas for $k = 10$ we get 10 folds where each fold is about 10% of the data set. This gives a training data of about 80% for $k = 5$ and 90% for $k = 10$ where each training set contains unique data, this makes the model more stable. From Figure 15 we see that for cross-validation using OLS as the linear regression method, there is not much of a difference between $k = 5$ and $k = 10$ and that they follow the same curve until degree 15 where the MSE curve for $k = 10$ dips while $k = 5$ rise. We also see that the standard deviation across the folds grows after degree 11. This matches Figure 13, where we see the variance of the model spike in the same range which points to an unstable model after degree 11. From this comparison we see that the best degree for cross-validated MSE is polynomial degree 11 in comparison to bootstrap at degree 8.

As mentioned earlier, bootstrap becomes more unstable at higher degrees since it only trains on about 63% of unique points of the training data. We also know that the cross-validation models evaluate the average over all data points, whereas the bootstrap can only evaluate over the 20% of data set aside for testing. The stableness of cross-validation in comparison to bootstrap can be supported by Figure 15. Despite this, we can see that for degree 1-8 the resampling techniques will follow the same curves, which points to the complexities of the models being roughly the same.

Using cross-validation with Ridge as the regression method, we can evaluate how the cross-validated MSE depends on both the degree of the polynomial and the penalty parameter λ (Table II). We see that the cross-validated MSE will decrease with the degree of polynomial and stay within the range of 2×10^{-2} after degree 6/7 when the best penalty parameter is very small. We see that the best penalty parameter depends on what degree, for very low degrees (i.e. 1-3), the best λ 's are very large. We see that for degree 1, the most optimal λ is the highest value it could be given the interval $[10^{-8}, 10^4]$. Since the degrees are so low, it means that they are very simple models with low variance, which is why they need little to no regularisation. Their cross-validated MSE is almost independent of the λ value. After this the penalty parameters will decrease to small, but non zero parameters that will still suppress unstable directions. We see that for both k values that the best degree is degree 12, instead of 11 as it was for OLS. The corresponding cross-validated MSE is $(2.068 \pm 0.54) \times 10^{-2}$ and $(1.918 \pm 0.78) \times 10^{-2}$ for Ridge compared to OLS with $(2.111 \pm 0.65) \times 10^{-2}$ and $(2.024 \pm 0.93) \times 10^{-2}$ for $k = 5$ and $k = 10$. This difference is very small and they're within the standard deviation of one another other. We can thus conclude that Ridge and OLS perform as good as each other when using k -fold cross-validation.

Figure 16 shows how the cross-validated MSE depends on the penalty parameter λ . For degrees 4 and 8, we see that the cross-validated MSE is constant for very small λ 's, this points to the degrees being stable (i.e. no need for regularisation). Considering the previous MSE graphs we've looked at for OLS, we know that the models are stable until degree 8. Which is why there is little need for regularisation for these degrees. After degree 8 however, the model becomes more complex and unstable. For degree 12 and 15, we see that the MSE is very high for small λ where the model overfits. The MSE then reaches a minimum at around $\lambda \approx 10^{-5}$. All the MSE will increase for larger λ and converge to about 0.1. We see that for low degrees the curves for $k = 5$ and $k = 10$ are almost identical. But for higher degrees $k = 10$ gives slightly lower than cross-validated MSE values.

E. Gradient descent code, with analytical gradients and automatic differentiation

We start by validating the analytical expressions we have for OLS and Ridge regression by comparing the gradients to the gradients found by using automatic differentiation. In table III we find that the difference between the two ways of calculating the gradient is very small, which indicated that the expressions we have, are correct. We therefore use these when we go forth with exploring gradient descent as an optimizer.

Equation E.1 describes how gradient descent searches for the minimum of the cost function $C(\theta)$. The learning rate γ decides the step length of each descent, and there will be an optimal step length which will ensure that we don't overshoot the minimum or spend too many iterations searching. In figure 17 we can clearly see that the maximum learning $\gamma_{\max}$ rate we calculated, is indeed the maximum. At this learning rate, the relative error between our gradient descent method and the closed form solution, shoots up. So to make sure our gradient descent method is valid, we need to make sure that we stay below this limit for the learning rate.

But we can also do even better. In figure 18 we study the number of iterations required at the different learning rates, and the best learning rate γ_{best} will be the one that required the minimum number of iterations to converge to the closed form solution. After γ_{best} is reached, we can see that the number of iterations required shoots up drastically. So even though the difference between the limit and the best learning rate is relatively small, the difference in optimization by using the different ones is quite large. In table IV, we can clearly see the difference between the two learning rates. For $\gamma_{\max}$, the minimum of the cost function is never reached within the set number of iterations for our program, so the result we're left with is just the value it reached within the range it was given. When we instead use γ_{best} we actually converge toward the closed form solution, and within a good margin of the iterations set.

We can further study the iterations needed for the solutions to converge, by looking at the difference between the GD solution and the closed form solution, for different values of the learning rate. This is what we've done in figure 19. We separately look at the convergence for OLS and Ridge, and can clearly see that for the learning rates we study here, the one that requires the least number of iterations is the best one we found. This is just verifying that the result we found in figure 18, is actually correct. We also plot for a learning rate higher than the limit value we calculated, and again this verifies that for a learning rate larger than this limit, the solution diverges.

We can now compare the cost functions for GD for both OLS and Ridge, and in figure 20 we do this. We include both the solution with the maximum learning rate and the best learning rate. For the maximum learning rate, the solutions never converge to the closed form so-

lutions within the iteration range it is given. They are instead cut-off when the iterations reach 10 000. When we instead use the best value for the learning rate, we actually converge to the closed form solutions within the given range, and the value reached is closer than it is with $\gamma_{\max}$.

We now need to test how well our model is by using it on data it hasn't been trained on, so the testing data set. In 21, we've plotted a histogram of the MSE for both OLS and Ridge, for the test data and the training data, using γ_{best} . In both cases we can see that the test MSE is slightly below the training MSE rather than above it, as we would've expected. For this problem, we're not looking at a lot of points, and the problem itself isn't that complex, so there is no over-fitting. Since the degree of the problem is a lot smaller compared to the number of points, the noise of the data will matter a lot more than the optimiser bias. So the fact that the MSE is smaller for the test data, can be purely due to the noise in the data, which is completely random.

For Ridge, we have a penalty factor λ that effects the outcome of the solution, and we can also study how this effects how well the model performs. In figure 22 we have the cost function as a function of the penalty λ for both the test data and the training data. At small penalty λ , the fit is barely constrained, and we see the same trend that is shown in figure 21, with the MSE of the training data being higher than for the test data. As the penalty increases, θ is restricted, so the MSE rises for both the test and training data.

F. Including momentum and more advanced ways to update the learning rate

With different gradient methods implemented, one could see how they were affected by the learning rates. In general models behaved similarly for both excess cost tolerances, with some managing to converge even above their estimated $\gamma_{\max}$. Why this occurs is unclear. Worth taking into account is that the results as seen in this report is tested for one dataset only, meaning one should be careful to generalize the behaviour to all model training. A further understanding could have been developed had we taken the time to compare results across different datasets.

Despite the quite limited testing of the different methods, it seems fair to claim that Adam does best in both cases. With the the broad range of learning rates, it manages to converge for the wide range. No other method manages to do so for both tolerances. This can likely be explained by Adam's RMS term, which works as a dampener of oscillations. In contrast, RMSProp does not converge for the tolerance of 10^{-8} at all likely due to this oscillation dampening effect where the step size probably decreased to the point that it would never be able to converge. Adam on the other hand with this term does, but likely due to the bias correction through the terms

$\hat{m}$ and $\hat{r}$.

For both tolerances, the Momentum method manages to converge the fastest for large steps. In a case such as ours with a relatively simple gradient landscape, this is to be expected. However, as the landscape becomes more complex, like in a neural network, this expectation that large steps yield low iterations for momentum might shatter. Adam, the combination of all the best of AdaGrad, RMSProp, and Momentum, might be the most stable and the best. Investigating this was not of priority, but would be interesting for further study.

G. Lasso regression

The comparison between gradient descent, coordinate descent and SciKit-learn’s solver, in table VI, shows that coordinate descent converges considerably faster for Lasso, with 181 iterations, than gradient descent does with 513 iterations. Both methods reach within 10^{-7} of the SciKit-learn reference. This is to be expected, as coordinate descent updates one coefficient at a time using a closed form soft threshold step, so each update can move directly to the optimal value for that coefficient. Whereas gradient descent must take small, fixed steps along an explicit sub gradient.

This explicit sub gradient is necessary because automatic differentiation (AD) does not recover the true sub gradient of $|\theta|$ at $\theta = 0$. As shown in table V, JAX’s AD return a single value of +1 for $\theta = 0$, which is an artefact of how `jnp.abs` is implemented, rather than a principled choice from the true sub gradient interval $[-1, 1]$. Using AD directly would bias updates for any coefficient sitting at or near zero, which is exactly the coefficient Lasso is designed to shrink toward. This is exactly why we implement the gradient descent method, in order to avoid errors that would be hard to detect.

In figure 24 we compare the cost function of the regression methods we have introduced: OLS, Ridge and Lasso. We include both gradient descent and coordinate descent for Lasso here, and see that they converge to the same solution. For all methods, Lasso is the method that converges in the fewest number of iterations, where coordinate descent is clearly the best one.

Table VII and figure 25 compare train and test MSE for OLS, Ridge and Lasso. All three methods achieve very similar test MSE (2.390×10^{-2} , 2.382×10^{-2} and 2.381×10^{-2} respectively) despite differing train MSE (2.561×10^{-2} , 2.677×10^{-2} and 2.688×10^{-2}). This suggests that for this dataset and polynomial degree, OLS is not over-fitting to begin with, so the variance reduction that regularisation provides does not translate into a meaningfully lower test error here. All three methods also show test MSE slightly below train MSE, consistent with the same pattern discussed in Part e, since this gap appears uniformly across all three methods rather than in only one, it is best explained by sampling variability in the train/test split rather than by any method-specific

issue.

In figure 27 we study the difference in how Ridge and Lasso coefficients behave as the penalty strength λ increases. Ridge shrinks all coefficients smoothly and continuously, but none of them reach exactly zero, even at large λ . Lasso drives the coefficients to exactly zero as λ grows, so its selecting the value of the coefficient exactly instead of shrinking toward it. This is the key qualitative difference between the two penalties, and explains why Lasso is often preferred when a sparse, more interpretable model is desired, at the cost of the closed-form solution and smooth optimisation landscape that Ridge retains.

H. Stochastic gradient descent

In our SGD method we start by varying the batch size, and study the effect this has on the cost function. In figure 28, we can easily see that for $M = 10$ the gradient estimate is a lot noisier for later iteration, than for batch sizes a bit larger. This is even clearer when we in figure 29 plot the batch size as a continuum. We can clearly see that there is a cut-off at just above $M = 10$, where the cost decreases drastically above this limit.

Next we study what happens when we change the learning rate γ , but keep i constant throughout the training of the data. In figure 30 we study three different learning rates, and clearly see that for a very low learning rate, the cost function is kept low. In figure 31, we have the learning rate as a continuum, and here it becomes even clearer where the cut-off for a stable learning rate is. At around $0.7 \cdot \gamma_{\max}$ there is again a very sharp increase in the cost.

As discussed in II, using a decaying schedule for the learning rate is beneficial in order to optimise the program. In figure 32 we’ve used the decaying schedule, where the initial value of the learning rate, is the same values we used in figure 30. The overall cost of this method is lower, and we can still see the same trends for size of the starting value compared to using that same starting value for the whole run of the program.

In figure 35, we’ve compared the decaying schedule to holding the learning rate constant. The overall cost is almost the same, and it is quite hard to tell the difference by only looking at the cost. So in the same figure, we’ve plotted the difference between them in order to see that there actually is a fluctuation between the cost of the two methods. In the same figure, we’ve also plotted the actual learning rate that is used throughout the training of the data. Here we can clearly see the decay of the size of the learning rate used for the decay schedule.

We compared the SGD method to the four adaptive optimisers we introduced in task f, and in figure 36, we’ve compared the cost functions for the different methods for both OLS and Ridge. The overall cost, for every method, is slight higher for Ridge than for OLS. For the momentum method, we needed to scale down the learning rate

γ , to avoid the method diverging. In table VIII, we have the values for how close to the closed form solutions the different methods came, where the best one was Ada-grad on Ridge, with a result of 1.52×10^{-3} . It also took the least number of iterations to reach this value. For RMSprop and Adam, on Ridge specifically, the methods underperform. This could be fixed by tuning the learning rate specifically for the different methods, rather than using the set value.

We compare the SGD method to the plain GD method we used in task e), by looking at the cost function of the two methods in figure 33. In table IX we have the values of the optimal parameters that the different methods converge to, and how many iterations it takes to get there. We can see that there is a slight difference for both OLS and Ridge in what optimal parameters they converge to. For SGD the table says that it takes 8001 iterations to converge for both OLS and Ridge, but this is not a true value. This just means that the program ran all the iterations it was set so, before the parameters could converge. Lastly we test the SGD model on the test data, and compare to the data the model was trained on. In figure 34 we have a histogram of the MSE for both OLS and Ridge for GD and SGD. In all cases, the MSE is lower for the test data than for the training data. This is not really to be expected, as the model should've performed worse on data its never seen before, than the data it was trained on. This could be due to small-sample noise in the splitting of the test and training data. If the MSE of the test data were a lot larger than the training data, it would indicate that the model is not very good. The ideal situation would be if the MSE was just slightly higher for the test data than the training data.

I. Cross-validation with hyperparameters

With k-fold cross-validation we were able to tune the hyperparameters such as the penalty λ and the polynomial degree to compare how Ridge, Lasso and OLS, and ultimately see which model was the best. In doing so several simplifications were made. When tuning the penalty, this was done on a fixed grid. Not can this be computationally expensive, but it also means that there are parameters λ that could have yielded better models. This was a compromise necessary to evaluate different methods within the time limit. The time restriction also meant investigating how different seeds impacted the cross-validation and additionally the attempted MSE-decomposition was not possible. Had we done this, we could have greater insight to how the different parameters affected the final result. As we knew the OLS is worse suited to more complicated data, investigating it was slightly neglected and its final fit was not investigated.

Through the training process, best models were selected. It is known that the training MSE of cross-validation tends to be optimistic [7], and that an indica-

tion for the models generalization ability has a correlation with how close the test MSE is to the train MSE shown in fig. 38. Close MSE's indicate that a model should be able to generalize better. This however does not quite line up with the final fits, both those of the best models and those of the cheapest best models figs. 40 and 41. In the case of the best fits, the model that is visually the closest to the true Runge function is are the Lasso fits, although their training and testing MSEs differ. As for the cheapest best fits, the model that comes visually the closest is the 5-fold Ridge fit, where the test and train MSE also differs. As per the rule of best cheapest models being within one standard deviation of the best score, such a result might actually be reasonable. Of course, the data used to train these models is also as shown noisy and sparse. Attributing the distance between the test and train MSE to be a total failure to the fits might be doing it an injustice. To properly asses this one would likely have to vary both seeds and number of generated data points, though in reality you would only have the data that you have, with no further datapoints available to assess. Additionally, visually verifying what fit is the best is probably not ideal. When we later approach multi-dimensional data to approach this becomes more difficult.

Interestingly, when we compare the Lasso fits across the best fits which have more polynomials vs the cheapest best ones, we see that Lasso does better for higher degrees. This is due to Lasso's hard penalty that disregards small features within the threshold of the penalty. On the lower polynomial fits featured in the cheaper models we see that this advantage dissipates and the Lasso fits give a similar result as the 10-fold Ridge fit, also of a lower degree.

Decomposing the MSE was somewhat a failure. Both bootstrap resampling and what we call k-fold resampling was attempted to do so initially only using the training set. The result (fig. 39) were decompositions that at times yielded errors on orders of magnitude higher than the test and train MSEs from the hyperparameter tuning process. Although as far as we understood it this was the right approach, the methods failed catastrophically in decomposing the MSE into its bias and variance counterparts. An attempt to decompose the MSE of the 5-fold Ridge with both resampling techniques was made as seen in section C. In resampling using the whole dataset the errors were mostly on the same scale as the train and test MSE. If we are to take this as the 'true' MSE decomposition, we can see that the bias(including noise) remains quite consistent for all numbers of resampling. The variance is really the deciding factor in the size of the error. Regardless of which data to use for the MSE decomposition, it seems reasonable to say that the variance is the dominating factor in deciding the size of the error across number of resamples. The issue of decomposing the MSE can be as discussed in section IVD can possibly be attributed to the instability for higher degrees.

V. CONCLUSION

For OLS we got the lowest test MSE at 1.871×10^{-2} at polynomial degree 8. After this the model became unstable and overfitted due to the design matrix becoming nearly linearly dependant at high degrees. Ridge regression removed this instability by shrinking the directions with small singular values. Using $\lambda = 0.040$, the MSE stayed stable throughout the degrees where OLS overfitted. The lowest test MSE for Ridge was 2.323×10^{-2} at degree 12. This is higher than that of OLS and shows that for this data set the main advantage of Ridge was the stability instead of a lower MSE score. The optimal Ridge model at degree 15 had 7.78 as the effective degrees of freedom. This is close to 8 which was the best degree for the OLS model and shows that both methods will give around the same model complexity.

The bias-variance analysis using bootstrap showed the expected behaviour. That models at low degrees are dominated by bias and models at high degrees are dominated by variance. Increasing the number of data points decreases the variance and moves the best degree towards more complex models. The best degree varied between 4-8 depending on which random seed was used, but for seed (i+1) it was 8.

k -fold cross-validation gave a more stable estimate of the MSE compared to bootstrap due to it being trained on more unique data points. Its best degree is 11 compared to bootstraps 8. For k -fold cross-validation, Ridge and OLS performed equally well. With best cross-validated MSE of $(2.111 \pm 0.65) \times 10^{-2}$ and $(2.068 \pm 0.65) \times 10^{-2}$ for $k = 5$ respectively and $(1.918 \pm 0.78) \times 10^{-2}$ and $(2.024 \pm 0.93) \times 10^{-2}$ for $k = 10$, respectively. The results for $k = 5$ and $k = 10$ were very similar.

For gradient descent (GD), validating our analytical gradients against automatic differentiation confirms our implementation was correct to within 10^{-16} . We found that the learning rate has an upper limit $\gamma_{\max}$ set by the curvature of the cost function, exceeding this limit will result in divergence. The number of iterations needed to converge is extremely sensitive to the learning rate value, even below this limit. The best learning rate we found, γ_{best} , converged within a few thousand iterations, while the limiting learning rate didn't converge within the iteration budget. This shows that just staying below $\gamma_{\max}$ is not enough to have the best optimization.

With gradient descent methods Momentum, AdaGrad, RMSProp and Adam implemented, we found that the

methods had different ranges of convergence for different tolerances. Occasionally methods exceeded their GD $\gamma_{\max}$. Though momentum was found to converge the fastest at a minimum of 1100 iterations for the largest convergent learning rate, Adam had over all lower iterations for convergence across the learning rate range ($10^{-3} - 7 \times 10^{-1}$).

For stochastic gradient descent, we found that the batch size, learning rate and choice of optimizer method, all have a comparable impact on convergence speed and final accuracy. Very small batch sizes give substantial gradient noise and increases the cost, while a decaying learning rate schedule reached the same final cost as a well tunes constant learning rate. If we cannot fine tune the constant learning rate, using a decaying schedule will be the preferred choice, as this will give us the optimal results. For the adaptive optimisers, Adagrad consistently performed best on this problem, converging fastest and closest to the closed form solution. Plain SGD did not reach the accuracy of standard gradient descent within the same iteration budget, which for this simple problem reflects the inherent trade-off between SGD's cheaper per-step cost and noisier convergence.

For the regression methods themselves, comparing OLS, Ridge and Lasso showed that all three Lasso solvers we implemented agreed closely with each other, with coordinate descent converging fastest. The key qualitative difference between Ridge and Lasso doesn't have to do with the predictive performance, but rather the structure of the solutions they produce. Ridge shrinks all the coefficients smoothly toward zero, without setting any of them exactly to zero, while Lasso drives coefficients exactly to zero as the penalty increases. Taken together, these results suggest that for this particular dataset, OLS already generalises about as well as Ridge or Lasso, so regularisation does not provide a substantial predictive advantage here. Lasso would still be the preferred method if a sparse, more interpretable model were a goal in itself, independent of predictive performance, since it is the only one of the three methods that performs explicit feature selection.

When assessing the best degree and penalty for models OLS, Ridge and Lasso using cross-validation hyperparameter tuning, it was found that the best models for OLS, Ridge and Lasso at 5 and 10 folds yielded and MSE of 2.1×10^{-2} to 2.7×10^{-2} , with Ridge 5-fold having the lowest at 2.1×10^{-2} with $\lambda = 3.56 \times 10^{-5}$. Using these best Ridge and Lasso models, well as their cheapest best models, we found that none of the fits managed to reproduce the Runge function.

-
- [1] V. Adamec, A brief history of AI and the rise of ChatGPT | GaussAlgo, <https://www.gaussalgo.com/knowledge-base/a-brief-history-of-ai-and-the-rise-of-chatgpt>.
 - [2] C. Stryker and E. Kavlakoglu, What is artificial intelligence (AI)? | IBM, <https://www.ibm.com/think/topics/artificial-intelligence>.
 - [3] D. Bergmann, What is machine learning? | IBM, <https://www.ibm.com/think/topics/machine-learning> (2025).

- [4] M. Hjorth-Jensen, *Machine Learning and the Physical Sciences* (Springer, 2026).
- [5] Interpolationcal, Runge's phenomenon: Why high-degree polynomials fail, <https://interpolationcalc.com/blog/runges-phenomenon/> (2026-06-17).
- [6] M. Hjorth-Jensen, Project 1 on machine learning, <https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/doc/Projects/2026/Project1/Project1.pdf> (2026).
- [7] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. (Springer, Ne, 2009).
- [8] Scikit-learn, 3.1. cross-validation: evaluating estimator performance, https://scikit-learn.org/stable/modules/cross_validation.html.
- [9] M. Hjorth-Jensen, Week 38: Automatic differentiation, adaptive learning rates and stochastic gradient descent, <https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/doc/WeeklyMaterial/week38/week38monday.pdf>.
- [10] Scikit learn, https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Lasso.html (2026).

Appendix A: Calculations

Deriving the bias-variance tradeoff for Part C: We want to show that

$$\mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] = \text{Bias}^2[\tilde{\mathbf{y}}] + \text{var}[\tilde{\mathbf{y}}] + \sigma^2.$$

For simpler notation we set $f(\mathbf{x}) = \mathbf{f}$, thus $\mathbf{y} = \mathbf{f} + \boldsymbol{\varepsilon}$. We add and subtract $\mathbb{E}[\tilde{\mathbf{y}}]$ which is the average prediction over training sets, this gives

$$\begin{aligned} \mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] &= \mathbb{E}[(\mathbf{f} + \boldsymbol{\varepsilon} - \tilde{\mathbf{y}})^2] \\ &= \mathbb{E}[(\mathbf{f} + \boldsymbol{\varepsilon} - \tilde{\mathbf{y}} + \mathbb{E}[\tilde{\mathbf{y}}] - \mathbb{E}[\tilde{\mathbf{y}}])^2] \\ &= \mathbb{E}\left[\left(\underbrace{\mathbf{f} - \mathbb{E}[\tilde{\mathbf{y}}]}_A + \underbrace{\mathbb{E}[\tilde{\mathbf{y}}] - \tilde{\mathbf{y}}}_B + \underbrace{\boldsymbol{\varepsilon}}_C\right)^2\right] \\ &= \mathbb{E}[A^2 + B^2 + C^2 + 2AB + 2AC + 2BC]. \end{aligned}$$

Since $\mathbf{f}$ is non-stochastic and $\mathbb{E}[\tilde{\mathbf{y}}]$ is already an expectation value, A is not random and can be taken outside the expectation. B is random through the training set, and C through the noise.

$$\begin{aligned} \mathbb{E}[2AB] &= 2A \mathbb{E}[\mathbb{E}[\tilde{\mathbf{y}}] - \tilde{\mathbf{y}}] = 2A \underbrace{(\mathbb{E}[\tilde{\mathbf{y}}] - \mathbb{E}[\tilde{\mathbf{y}}])}_0 = 0 && (\mathbf{f} \text{ non-stochastic}) \\ \mathbb{E}[2AC] &= 2A \underbrace{\mathbb{E}[\boldsymbol{\varepsilon}]}_0 = 0 && (\text{zero-mean noise}) \\ \mathbb{E}[2BC] &= 2 \mathbb{E}[B] \underbrace{\mathbb{E}[\boldsymbol{\varepsilon}]}_0 = 0 && (\text{noise independent of training set}) \end{aligned}$$

For the last term, independence lets the expectation of the product factorize into the product of expectations, and the zero-mean noise makes it vanish.

Cross terms have now vanished and we are left with

$$\mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] = \mathbb{E}[(\mathbf{f} - \mathbb{E}[\tilde{\mathbf{y}}])^2] + \mathbb{E}[(\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}])^2] + \mathbb{E}[\boldsymbol{\varepsilon}^2],$$

where we used $(\mathbb{E}[\tilde{\mathbf{y}}] - \tilde{\mathbf{y}})^2 = (\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}])^2$. Since the noise has zero mean, $\mathbb{E}[\boldsymbol{\varepsilon}^2] = \text{Var}[\boldsymbol{\varepsilon}] + \mathbb{E}[\boldsymbol{\varepsilon}]^2 = \sigma^2$. Writing the expectation values as sample means, we obtain

$$\mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] = \frac{1}{n} \sum_i (f_i - \mathbb{E}[\tilde{\mathbf{y}}])^2 + \frac{1}{n} \sum_i (\tilde{y}_i - \mathbb{E}[\tilde{\mathbf{y}}])^2 + \sigma^2,$$

which is exactly

$$\mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] = \text{Bias}^2[\tilde{\mathbf{y}}] + \text{var}[\tilde{\mathbf{y}}] + \sigma^2,$$

with

$$\begin{aligned} \text{Bias}^2[\tilde{\mathbf{y}}] &= \mathbb{E}[(\mathbf{f} - \mathbb{E}[\tilde{\mathbf{y}}])^2] = \frac{1}{n} \sum_i (f_i - \mathbb{E}[\tilde{\mathbf{y}}])^2, \\ \text{Variance}[\tilde{\mathbf{y}}] &= \mathbb{E}[(\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}])^2] = \frac{1}{n} \sum_i (\tilde{y}_i - \mathbb{E}[\tilde{\mathbf{y}}])^2. \end{aligned}$$

Appendix B: Use of AI/LLM tools

1. Tools used.

Claude (claude.ai, September 2026): Models

- Model: Sonnet 5, Effort: Medium until September 27 as Sonnet 5.5 launched September 28.
- Model: Sonnet 5.5, Effort: Medium September 28-31.
- Model: Opus 5.5 Effort: Medium October 1st - 5th.

2. Text writing and editing.

Section	Level	Notes
Abstract	1	LLM helped with restructuring sentences to shorten down the abstract.
Introduction	1	LLM helped with spellchecking and restructuring sentences from ideas we supplied.
Methods	2	LLM rewrote and restructured and sentences from ideas we supplied. LLM was used as an academic sparring partner to better understand unclear concepts from the material provided
Results	1	Output data from code was sent into LLM to convert to Table format. Figures were sent into LLM to convert into latex syntax.
Discussion	2	LLM was used as an academic sparring partner when interpreting results to see if they were correct with the theory from methods. LLM was also used to help with restructuring sentences that were hard to read.
Conclusion	1	Language corrections

3. Code generation and assistance.

File / notebook	Level	Description
<code>__init__.py</code>	1	How to access modules from other files
<code>_Part_a.py</code>	1	LLM assisted with how to plot the heat map and how to properly extract the lowest MSE and highest R2 score from the heatmap. LLM also checked if independently written code worked as expected. No changes were made.
<code>_Part_b.py</code>	1	LLM assisted with how to plot the heat map and how to properly extract the lowest MSE and highest R2 score from the heatmap. LLM also checked if independently written code worked as expected. No changes were made.
<code>_Part_c.py</code>	1	LLM checked if independently written code worked as expected. No changes were made.
<code>_Part_d.py</code>	2	LLM assisted in plotting the error bars for cross validation and how to print the output for bootstrap and CV lowest MSE. LLM assisted with for loop to print the values that make up Table II. LLM assisted with for loop to plot Figure 16. LLM also checked if independently written code worked as expected. LLM notified that maxdegree had changed in our code compared to cross_validation default, this was fixed by us.
<code>_Part_e.py</code>	1	LLM helped with debugging code, fixed minor error that didn't let code run.
<code>_Part_f.py</code>	1	LLM used for minor debugging
<code>_Part_g.py</code>	2	LLM helped with the comparison of different Lasso methods.
<code>_Part_h.py</code>	2	LLM looked over code, and flagged that sgd for the adaptive optimisers hadn't been done yet, and showed a suggestion to how this could be implemented.
<code>_Part_i.py</code>	2	Writing plotting plot_best_model.fits
<code>colors.py</code>	1	Colors throughout the program were manually written down by hex codes, LLM provided the name of the color for these hex codes.
<code>general_functions.py</code>	1	All functions in file were written by us. LLM was used to check if the code worked as expected. No changes were made.
<code>gradient_descent_methods.py</code>	1	LLM helped debug and comment code.
<code>lasso_methods.py</code>	1	LLM helped debug and comment code.
<code>main_test.py</code>	2	How to access modules from other files. Cleaning up visualisation of how n resamples using kfold resampling affects the bias variance decomposition
<code>optimizer_methods.py</code>	1	LLM helped debug and comment code.
<code>plots.py</code>	4	LLM wrote all the functions in this file after being provided with information of what we wanted to plot and how. Functions were commented manually but with basis in the few comments provided by the LLM.
<code>README.md</code>	2	Asked LLM to make a structure of the program folders and help with syntax when writing the README.
<code>regression_methods.py</code>	1	All functions in file were written by us. LLM was used to check if the code worked as expected. No changes were made
<code>requirements.txt</code>	4	Asked LLM to create a text file that contained the requirements needed to run the code in Project 1
<code>resampling_methods.py</code>	3	K-fold resampling algorithm mostly LLM made Implemented test_wholedataset function in bootstrap_resampling based on selfmade version in kfold_resampling. Ran through for glaring bugs Assisted with doc strings for train_through_gridsearchCV, mse.decomposer and grid_search impementation of std calculations in other methods than cross_validation, to mirror usage already defined there
<code>sgd_methods.py</code>	1	LLM helped debug and comment code.

Appendix C: More Figures

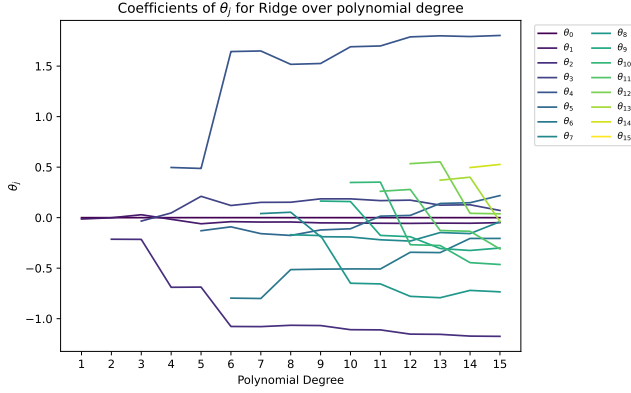


Figure 42: Same data as in Figure 7 but with penalty parameter 0.040. Visualization of how the penalty parameters suppresses the larger coefficients as all θ_j stay within $[-1.0, 1.5]$

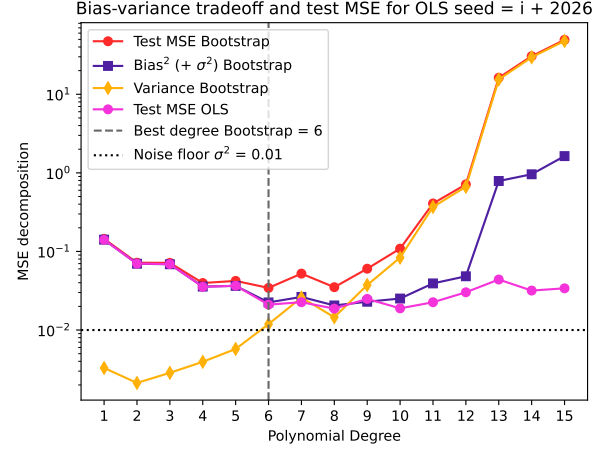


Figure 44: Random state with seed = $i + 2026$ in resampling for bootstraps changes the curves in the MSE decomposition and the optimal degree moves to degree 6

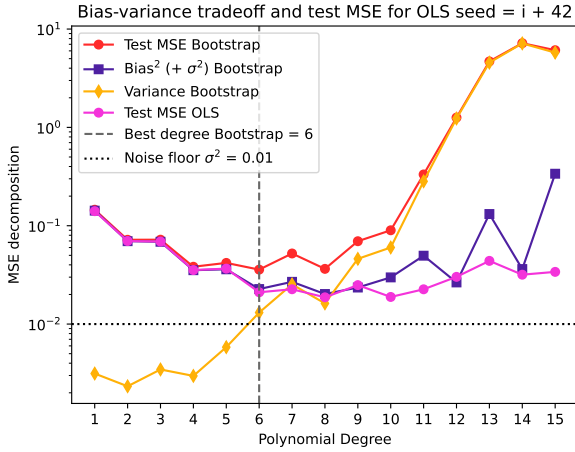


Figure 43: Random state with seed = $i + 42$ in resampling for bootstraps changes the curves in the MSE decomposition and the optimal degree moves to degree 6

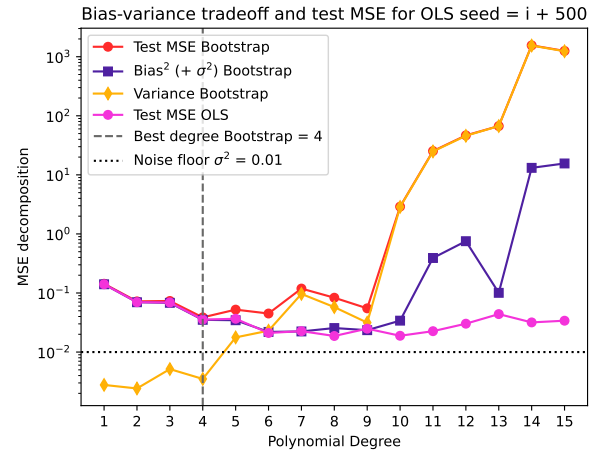


Figure 45: Random state with seed = $i + 500$ in resampling for bootstraps changes the curves in the MSE decomposition and the optimal degree moves to degree 4

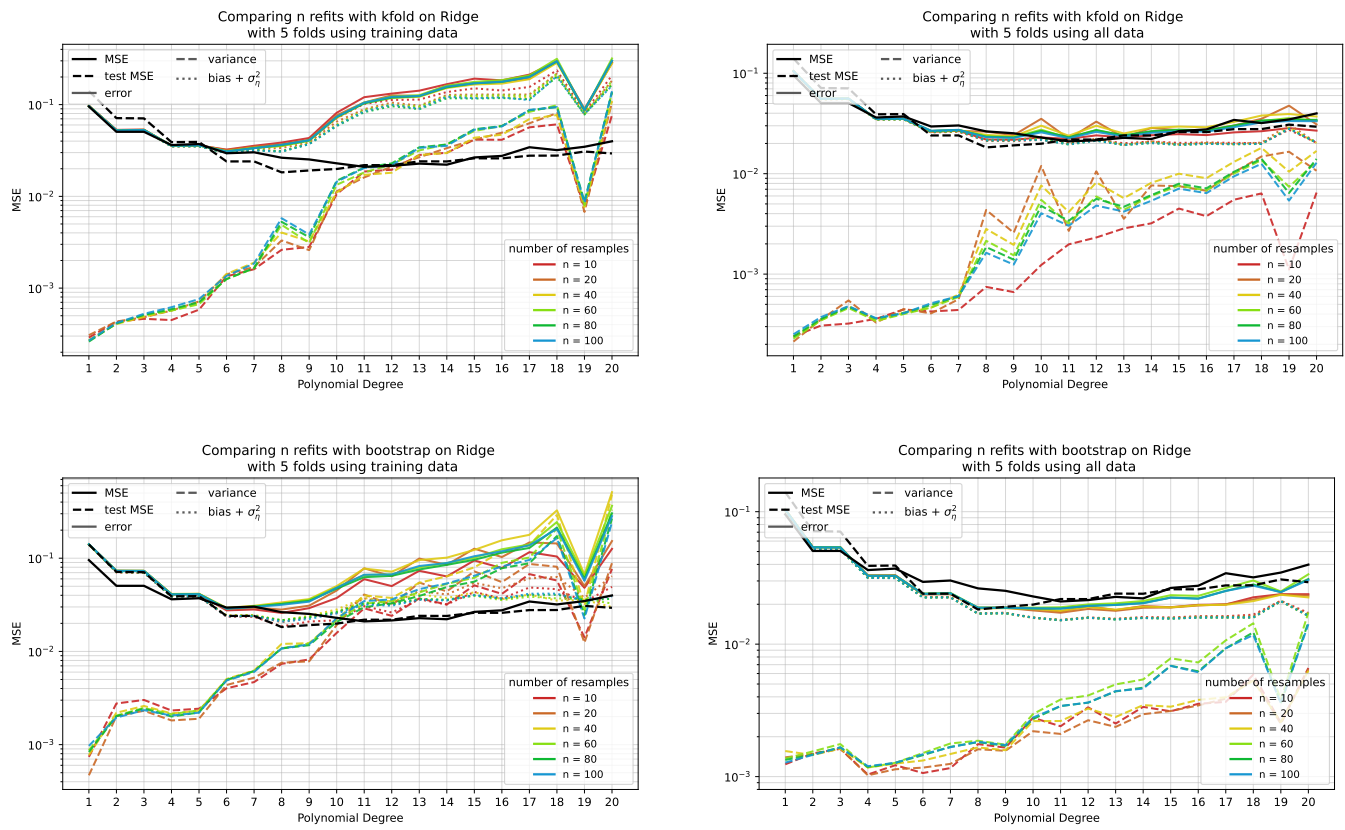


Figure 46: Plots on the right show MSE decomposition using resampling on the training set, while those on the left show resampling on the whole set. Proper decomposition uses the same data as the training MSE gives, but this yields a terrible decomposition. An attempt was made using the whole training set. These match the training and testing MSEs better, but include more data than both have available. Clearly this MSE decomposition strategy was unfitting. The usage can be found in `Tests_and_additional_results/main_test.ipynb` with `Code/resampling_methods.py` showing implementation of methods.